



西安电子科技大学
XIDIAN UNIVERSITY

第三章 运算方法与运算器

西安电子科技大学
人工智能学院
林 杰



计算机中的运算

- 高级语言程序中涉及的运算（以C语言为例）
 - 逻辑位运算： $|$ (OR), $\&$ (AND), $\sim$ (NOT), $\wedge$ (XOR)
 - 关系运算： $||$ (OR), $\&\&$ (AND), $!$ (NOT)
 - 移位运算： $x \ll k$, $x \gg k$
 - 算术运算 “+、-、*、/”
- 这些运算操作符都会编译成底层的逻辑运算指令或算术运算指令
 - 逻辑位运算会直接翻译成逻辑或、与、非等逻辑操作指令
 - 关系运算会翻译成各种条件分支代码
 - 移位运算会翻译成算术或逻辑移位指令
 - 算术运算则会根据符号的数据类型翻译成定点或浮点运算指令



- 指令集中涉及的运算（如RISC-V指令系统提供的运算类指令）
 - 涉及的定点数运算
 - 算术运算
 - 带符号整数：取负/符号扩展/加/减/乘/除/算术移位
 - 无符号整数：0扩展/加/减/乘/除
 - 逻辑运算
 - 逻辑操作：与/或/非/...
 - 移位操作：逻辑左移/逻辑右移
 - 涉及的浮点数运算：加、减、乘、除

**所有运算都可由
加法器或ALU+移位器+多路选择器+控制逻辑
实现！**



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器
2. 浮点运算器



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器
2. 浮点运算器

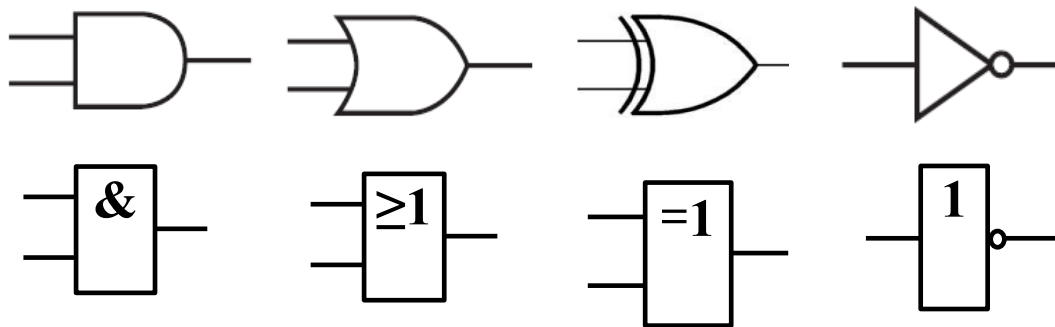


一. 逻辑与移位运算

1. 机器数的逻辑运算

- 基本逻辑运算：与、或、异或、反相
- 特点：按位操作

X_i	Y_i	$X_i \cdot Y_i$	$X_i + Y_i$	$X_i \oplus Y_i$	$\bar{X}_i$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0





2. 机器数的移位运算

1) 移位的意义 $15.\text{m} = 1500.\text{cm}$ 小数点右移 2 位

机器用语: 15 相对于小数点 左移 2 位
(小数点不动)

左移——绝对值扩大 右移——绝对值缩小

- 在计算机中, 移位与加减配合, 能够实现乘除运算
- 计算机的移位运算分为逻辑移位、算术移位、循环移位三种。主要区别在于符号位和移出的数据位的处理方法不同。



2) 算术移位

算术移位后**符号位不变**，具体的移位规则如表所示。

不同机器数算术移位后得空位填补规则***

数据类型	码制	添补代码
正数	原码、补码、反码	0
负数	原码	0
	补码	右移添1
		左移添0
	反码	1





2) 算术移位

【例3.1】设机器数字长为8位（含一位符号位），写出 $A=+26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A=+26=+11010$ ， $[A]_{\text{原}}=[A]_{\text{补}}=[A]_{\text{反}}=0,0011010$

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}}=[A]_{\text{补}}=[A]_{\text{反}}$	
移位前	0,0011010	+26
左移一位	0,011010 0	+52
左移两位	0,11010 00	+104
右移一位	0, 0 001101	+13
右移两位	0, 00 00110	+6



2) 算术移位

【例3.2】设机器数字长为8位（含一位符号位），写出 $A=-26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A=-26=-11010$

原码移位结果

移位操作	机 器 数	对应的真值
移位前	1,0011010	- 26
左移一位	1,0110100	- 52
左移两位	1,1101000	- 104
右移一位	1,0001101	- 13
右移两位	1,0000110	- 6



2) 算术移位

【例3.2】设机器数字长为8位（含一位符号位），写出 $A=-26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A=-26=-11010$

补码移位结果

移位操作	机 器 数	对应的真值
移位前	1,1100110	- 26
左移一位	1,1001100	- 52
左移两位	1,0011000	- 104
右移一位	1,1110011	- 13
右移两位	1,1111001	- 6



2) 算术移位

【例3.2】设机器数字长为8位（含一位符号位），写出 $A=-26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A=-26=-11010$

反码移位结果

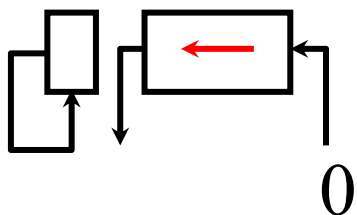
移位操作	机 器 数	对应的真值
移位前	1,1100101	- 26
左移一位	1,100101 1	- 52
左移两位	1,00101 11	- 104
右移一位	1, 1 110010	- 13
右移两位	1, 11 11001	- 6



一. 逻辑与移位运算

2) 算术移位

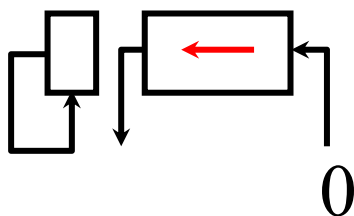
数据类型	码制	添补代码
正数	原码、补码、反码	0
负数	原码	0
	补码	右移添1
		左移添0
	反码	1



(a) 真值为正

← 丢 1 出错

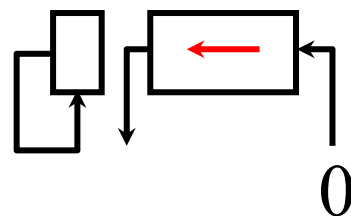
→ 丢 1 影响精度



(b) 负数的原码

出错

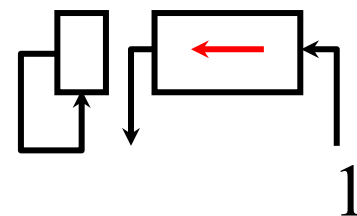
影响精度



(c) 负数的补码

正确

影响精度



(d) 负数的反码

正确

正确



一. 逻辑与移位运算

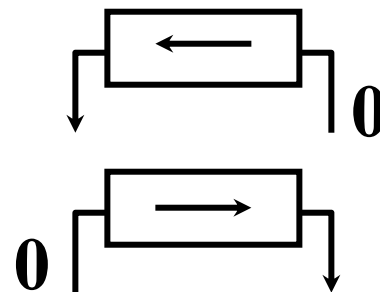
3) 逻辑移位

算术移位： 有符号数的移位

逻辑移位： 无符号数的移位

逻辑左移 低位添 0，高位移丢

逻辑右移 高位添 0，低位移丢



例如

01010011

10110010

逻辑左移

1010011**0**

逻辑右移

01011001

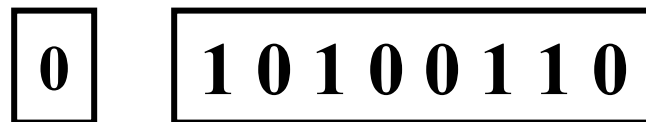
算术左移

0010011**0**

算术右移

11011001 (补码)

高位 1 移丢



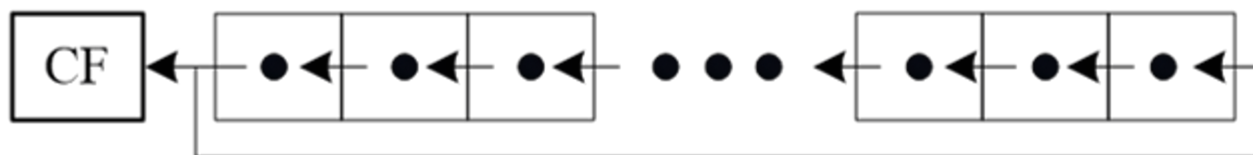


一. 逻辑与移位运算

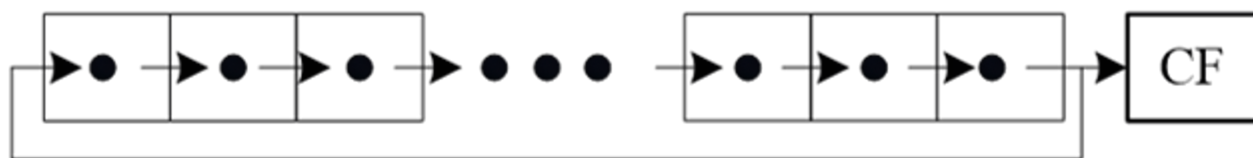
4) 循环移位

循环移位分为带进位标志位CF的循环移位（大循环）和不带进位标志位的循环移位（小循环）。

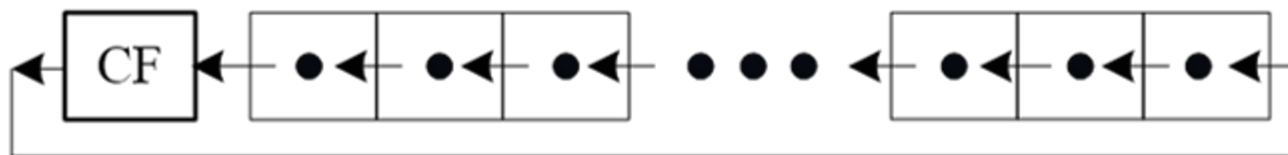
不带进位
循环左移



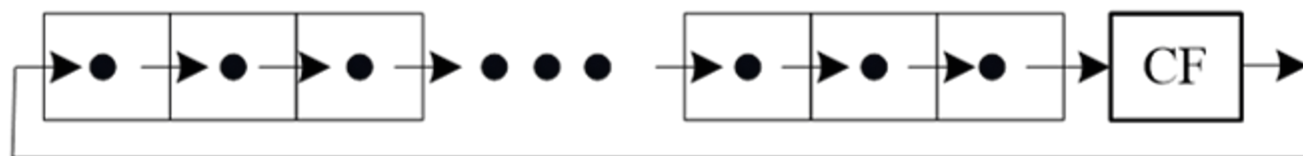
不带进位
循环右移



带进位循
环左移



带进位循
环右移





3. 带符号数的舍入操作

- 在算术右移时，由于受到硬件的限制，运算结果有可能需要补上一定的尾数，这样会造成一些误差。为了缩小误差，要进行舍入处理。
- 假定运算的结果 $p+q$ 位，保留前 p 位。
- 常见的舍入方法有：
 - ①恒舍(切断)。实现简单，但误差大。
 - ②冯.诺依曼舍入法，又称为恒置1法，把保留部分 p 位的最低位置1。实现简单，平均误差接近0，应用较多。



③下舍上入法，即0舍1入，相当于十进制的四舍五入。

用 q 位的最高位作为判断标志，以决定保留部分是否加1。

- 如该位为0，则舍去整个 q 位，相当于恒舍；
- 如该位为1，则在保留的 p 位的最低位上加1。

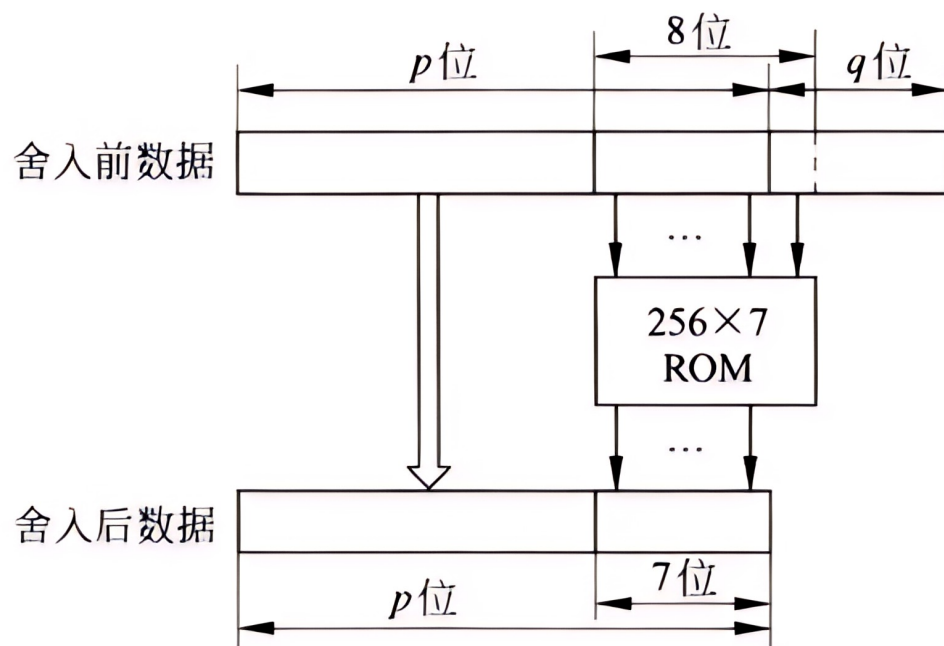
该方法在降低误差上有很大进步，但需要增加硬件做加法舍入。

目前较少使用，主要用在软件实现的浮点算法中。



一. 逻辑与移位运算

④查表舍入法，或ROM舍入法，每次经查表来读得相应的处理结果。ROM查表舍入法是将 k 位数据下溢处理成 $k-1$ 位的结果。表的设计通常是当尾数最低 $k-1$ 位全1时用截断法，其余用舍入法。实现速度快，虽然增加了硬件，但是整体性能最佳。



查表舍入法的原理

8位 → 7位

地址 内容

000 00

001 01

010 01

011 10

100 10

101 11

110 11

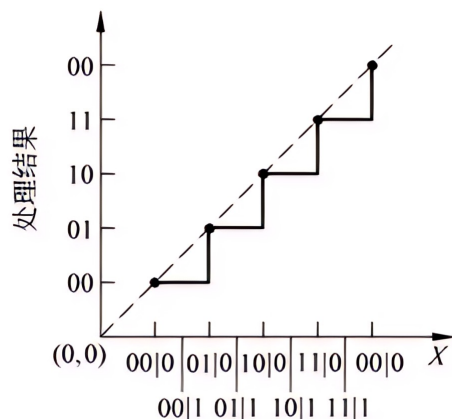
111 11

3位 → 2位

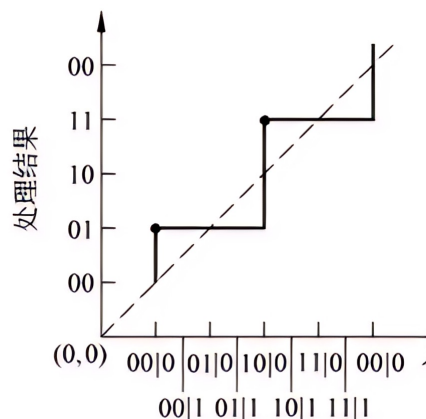


一. 逻辑与移位运算

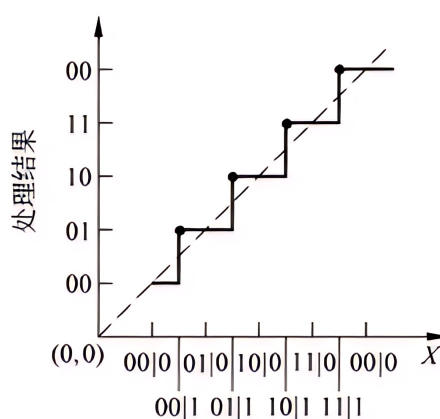
	恒舍法	恒置1法	舍入法	查表舍入法
优势	实现简单，不增加硬件，不需要处理时间	实现简单，不增加硬件，不需要处理时间，平均误差趋于零。	实现简单，增加的硬件少，最大误差小，平均误差接近零	速度快，平均误差可调节到零。
缺点	平均误差大且无法调节	最大误差最大	处理速度慢，最坏的情况下可能需要从尾数的最低位进位至最高位。	需要的硬件量大。



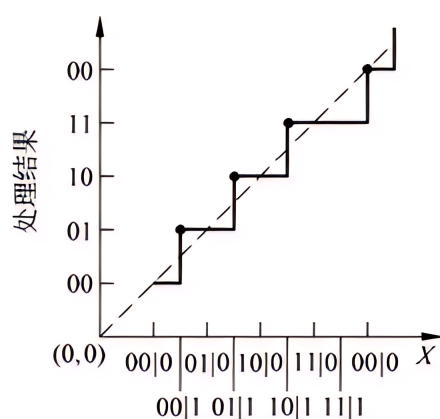
恒舍法



恒置1法



舍入法



查表舍入法

各种舍入处理方法的误差曲线





一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器
2. 浮点运算器



二. 定点数加减运算

1. 补码加减法运算

(1) 补码加法 $[X]_{\text{补}} + [Y]_{\text{补}} = [X + Y]_{\text{补}} \pmod{M} \quad (3-1)$

即：在以 M 为模时，两数补码的和等于两个数和的补码。

对定点小数： $M=2$ ；对定点整数来说， $M=2^{n+1}$

证明：以定点小数为例。

根据补码的定义， $M=2$ ， $-1 \leq X < 1$ ， $-1 \leq Y < 1$ ，且 $-1 \leq X+Y < 1$ （无溢出）。

分以下四种情况：

(1) 若 $X > 0$ ， $Y > 0$ ，则 $X+Y > 0$ 。

由于正数补码与真值相同，可知： $[X]_{\text{补}} = X$ ； $[Y]_{\text{补}} = Y$

所以

$$[X]_{\text{补}} + [Y]_{\text{补}} = X + Y = [X + Y]_{\text{补}}$$



二. 定点数加减运算

(2) 若 $X>0$, $Y<0$

相加的结果有正、负两种可能。

根据补码的定义：

$$[X]_{\text{补}}=X; \quad [Y]_{\text{补}}=2+Y; \quad [X]_{\text{补}}+[Y]_{\text{补}}=2+(X+Y)$$

- 当 $X+Y>0$, $2+X+Y>2$, 2为模数, 可直接舍弃, 故

$$[X]_{\text{补}}+[Y]_{\text{补}}=2+(X+Y)=X+Y=[X+Y]_{\text{补}} \quad (X+Y>0)$$

- 当 $X+Y<0$, 将 $X+Y$ 看成一个负数, 根据补码的定义可得:

$$[X]_{\text{补}}+[Y]_{\text{补}}=2+(X+Y)=[X+Y]_{\text{补}} \quad (X+Y<0)$$

(3) 若 $X<0$, $Y>0$

这种情况和情况 (2) 对称, 无须单独证明。



二. 定点数加减运算

(4) 若 $X < 0$, $Y < 0$, 则 $-2 \leq X+Y < 0$

根据补码的定义:

$$[X]_{\text{补}} = 2 + X; \quad [Y]_{\text{补}} = 2 + Y;$$

• 所以:

$$[X]_{\text{补}} + [Y]_{\text{补}} = 2 + 2 + X + Y = 2 + (2 + X + Y)$$

• 由于 $X+Y$ 为负数, $-2 \leq X+Y < 0$, 因此, $0 \leq 2+X+Y < 2$, 根据补码定义:

$$[X]_{\text{补}} + [Y]_{\text{补}} = 2 + (2 + X + Y) = 2 + X + Y \pmod{2}$$

• 又因为 $X+Y < 0$, 根据负数补码定义有:

$$[X]_{\text{补}} + [Y]_{\text{补}} = 2 + X + Y = [X+Y]_{\text{补}} \quad (X+Y < 0)$$

综上所述, 对于定点小数, 式(3-1) 成立。

同理, 也可以证明该公式也同样适用于整数。



二. 定点数加减运算

【例3.3】 设 $A = 0.1011$, $B = -0.0101$

求 $[A + B]_{\text{补}}$

验证

解: $[A]_{\text{补}} = 0.1011$

$+ [B]_{\text{补}} = 1.1011$

$[A]_{\text{补}} + [B]_{\text{补}} = 10.0110 = [A + B]_{\text{补}}$

$\therefore A + B = 0.0110$

$$\begin{array}{r} 0.1011 \\ - 0.0101 \\ \hline 0.0110 \end{array}$$

【例3.4】 设 $A = -9$, $B = -5$

求 $[A+B]_{\text{补}}$

验证

解: $[A]_{\text{补}} = 1, 0111$

$+ [B]_{\text{补}} = 1, 1011$

$[A]_{\text{补}} + [B]_{\text{补}} = 11, 0010 = [A + B]_{\text{补}}$

$\therefore A + B = -1110$

$$\begin{array}{r} -1001 \\ + -0101 \\ \hline -1110 \end{array}$$



(2) 补码减法

$$[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} \pmod{M} \quad (3-2)$$

已知 $[Y]_{\text{补}}$ ，求 $[-Y]_{\text{补}}$

解：以小数为例，设 $[Y]_{\text{补}} = Y_0.Y_1Y_2\dots Y_n$

第一种情况： $[Y]_{\text{补}} = 0.Y_1Y_2\dots Y_n$

$$\therefore Y = 0.Y_1Y_2\dots Y_n; \quad -Y = -0.Y_1Y_2\dots Y_n$$

$$\therefore [-Y]_{\text{补}} = 1.\overline{Y_1}\overline{Y_2}\dots\overline{Y_n} + 0.00\dots 1 = [[Y]_{\text{补}}]_{\text{求补}}$$

第二种情况： $[Y]_{\text{补}} = 1.Y_1Y_2\dots Y_n$

$$\therefore [Y]_{\text{原}} = 1.\overline{Y_1}\overline{Y_2}\dots\overline{Y_n} + 0.00\dots 1$$

$$Y = -(0.\overline{Y_1}\overline{Y_2}\dots\overline{Y_n} + 0.00\dots 1)$$

$$\therefore -Y = (0.\overline{Y_1}\overline{Y_2}\dots\overline{Y_n} + 0.00\dots 1)$$

$$\therefore [-Y]_{\text{补}} = 0.\overline{Y_1}\overline{Y_2}\dots\overline{Y_n} + 0.00\dots 1 = [[Y]_{\text{补}}]_{\text{求补}}$$

求补：所有位(含符号)取反，末位加1。



补码加减运算规则：

- ①参加运算的操作数用补码表示；
- ②补码的符号位与数值位同时进行加运算。
- ③ 加：两数补码直接相加；
 减：减数补码连同符号位一起按位取反，末位加1；
 再与被减数的补码相加。
- ④ 运算结果即为和/差的补码。



二. 定点数加减运算

【例3.5】 设机器数字长为 8 位（含 1 位符号位）
且 $A = 15$, $B = 24$, 用补码求 $A - B$

解: $A = 15 = 0001111$ $B = 24 = 0011000$

$$\begin{array}{r} [A]_{\text{补}} = 0, 0001111 \quad [B]_{\text{补}} = 0, 0011000 \\ + [-B]_{\text{补}} = 1, 1101000 \\ \hline \end{array}$$

$$[A]_{\text{补}} + [-B]_{\text{补}} = 1, 1110111 = [A - B]_{\text{补}}$$

$$\therefore A - B = -1001 = -9$$



(3) 符号扩展

- 将给定位数表示的数转换成具有更多位数的某种表示形式。
如8位扩展至16位。
- 正数：符号位不变，其他附加位都用0进行填充。
- 负数：根据机器数的不同而扩展方式不同。
 - 原码：符号位不变，其他附加位都用0进行填充。
 - 补码：符号位不变，所有附加位都用1进行填充。
- 综上所述，补码的符号扩展，所有附加位均用符号位填充，
即正数用0填充，负数用1填充。



二. 定点数加减运算

【例3.6】 $X=11, Y=7$, 用补码求 $X+Y=?$



$$[X]_{\text{补}} = 0, 1011, \quad [Y]_{\text{补}} = 0, 0111$$

$$\begin{array}{r} 0, 1011 \\ + \quad 0, 0111 \\ \hline 1, 0010 \end{array}$$

$$[X+Y]_{\text{补}} = 1, 0010$$

$$X+Y = -1110 = -(14)_{10}$$

【例3.7】 $X=-11, Y=-7$, 用补码求 $X+Y=?$

$$[X]_{\text{补}} = 1, 0101, \quad [Y]_{\text{补}} = 1, 1001$$

$$\begin{array}{r} 1, 0101 \\ + \quad 1, 1001 \\ \hline 10, 1110 \end{array}$$

$$[X+Y]_{\text{补}} = 0, 1110$$

$$X+Y = 1110 = (14)_{10}$$



2. 补码的溢出判断与检出方法

(1) 溢出的产生

- 当运算结果超出机器数所能表示的范围时称为溢出。
 - 两个异号数相加或两个同号数相减，其结果不会溢出。
 - 两个同号数相加或两个异号数相减，才有可能发生溢出。
 - 两正数相加产生的溢出称为正溢；
 - 两负数相加产生的溢出称为负溢。



(2) 溢出检测方法

假设，被操作数： $[X]_{\text{补}} = X_s, X_1 X_2 \dots X_n$;

操作数为： $[Y]_{\text{补}} = Y_s, Y_1 Y_2 \dots Y_n$

其和(差)为： $[S]_{\text{补}} = S_s, S_1 S_2 \dots S_n$

$$\begin{array}{r} 0,1011 \\ + 0,0111 \\ \hline 1,0010 \end{array}$$

① 采用一位符号位

• 当 $X_s = Y_s = 0$, $S_s = 1$, 产生正溢;

$$\begin{array}{r} 1,0101 \\ + 1,1001 \\ \hline 10,1110 \end{array}$$

• 当 $X_s = Y_s = 1$, $S_s = 0$, 产生负溢;

• 溢出判断条件为: $OF = X_s \cdot Y_s \cdot \overline{S_s} + \overline{X_s} \cdot \overline{Y_s} \cdot S_s$ (3-3)



二. 定点数加减运算

② 采用进位法

- 两数运算时，产生的进位为：

$$\begin{array}{r} \nwarrow \\ 0,1011 \\ + \quad 0,0111 \\ \hline 1,0010 \end{array}$$

$$\begin{array}{r} \nwarrow \\ 1,0101 \\ + \quad 1,1001 \\ \hline 10,1110 \end{array}$$

$$C_s, C_1 C_2 \dots C_n$$

其中， C_s 为符号位产生的进位， C_1 为最高数值位产生的进位。

- 两正数相加，当最高有效位产生进位($C_1 = 1$)而符号位不产生进位($C_s = 0$)时，发生正溢；
- 两负数相加，当最高有效位不产生进位($C_1 = 0$)而符号位产生进位($C_s = 1$)时，发生负溢；
- 溢出判断条件为：

$$OF = \overline{C_s} C_1 + C_s \overline{C_1} = C_s \oplus C_1 \quad (3-4)$$



二. 定点数加减运算

③ 采用变形补码（双符号位补码）

- 将符号位扩充至两位 S_{s_1} 和 S_{s_2} ，其所能表示的信息随之扩大，既能检测出是否溢出，又能指出结果的符号。
- 在双符号位的情况下，左边的符号位 S_{s_1} 为真符，两个符号位都作为数的一部分参加运算。

• 双符号位的含义

- $S_{s_1}S_{s_2} = 00$ 结果为正数，无溢出

- $S_{s_1}S_{s_2} = 01$ 结果正溢

- $S_{s_1}S_{s_2} = 10$ 结果负溢

- $S_{s_1}S_{s_2} = 11$ 结果为负数，无溢出

- 溢出判断条件为： $OF = S_{s_1} \oplus S_{s_2}$ (3-5)


$$\begin{array}{r} 00,1011 \\ + 00,0111 \\ \hline 01,0010 \end{array}$$


$$\begin{array}{r} 11,0101 \\ + 11,1001 \\ \hline 110,1110 \end{array}$$



二. 定点数加减运算

3. 加减法的逻辑实现

(1) 一位全加器 (Full-Adder)

- 输入 X_i 和 Y_i ，低位对该位的进位为 C_i
- 本位和 S_i ，向高位的进位 C_{i+1}

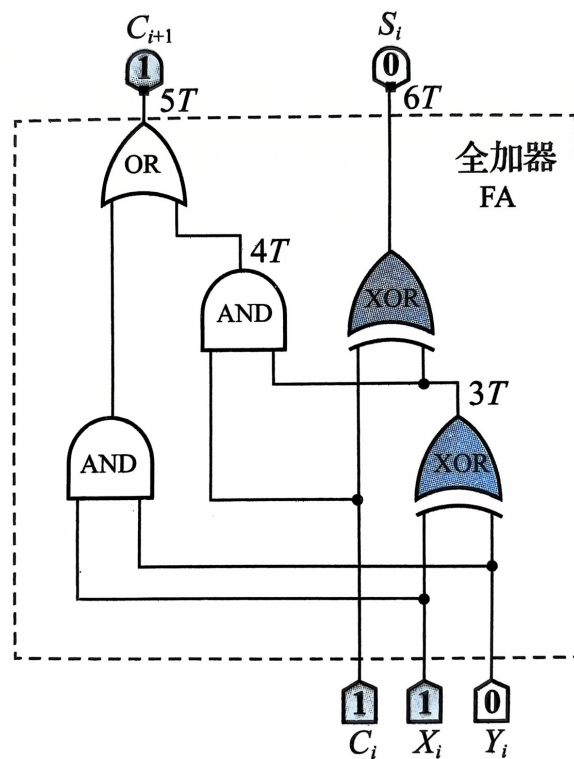
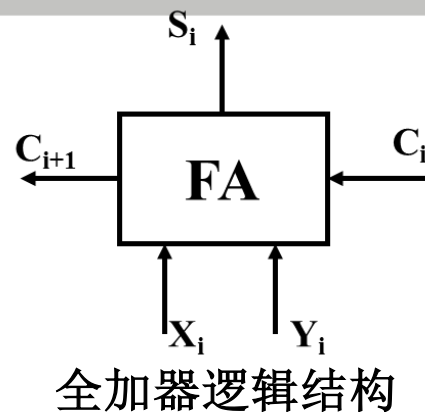
一位全加器真值表				
X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

假设传播延迟：
一级门（如与，
或门，与非）为 T
异或门 $3T$

- 根据真值表，得全加器逻辑表达式

$$S_i = X_i \oplus Y_i \oplus C_i \quad (3-6a)$$

$$C_{i+1} = (X_i \cdot Y_i) + (X_i \oplus Y_i) \cdot C_i \quad (3-6b)$$



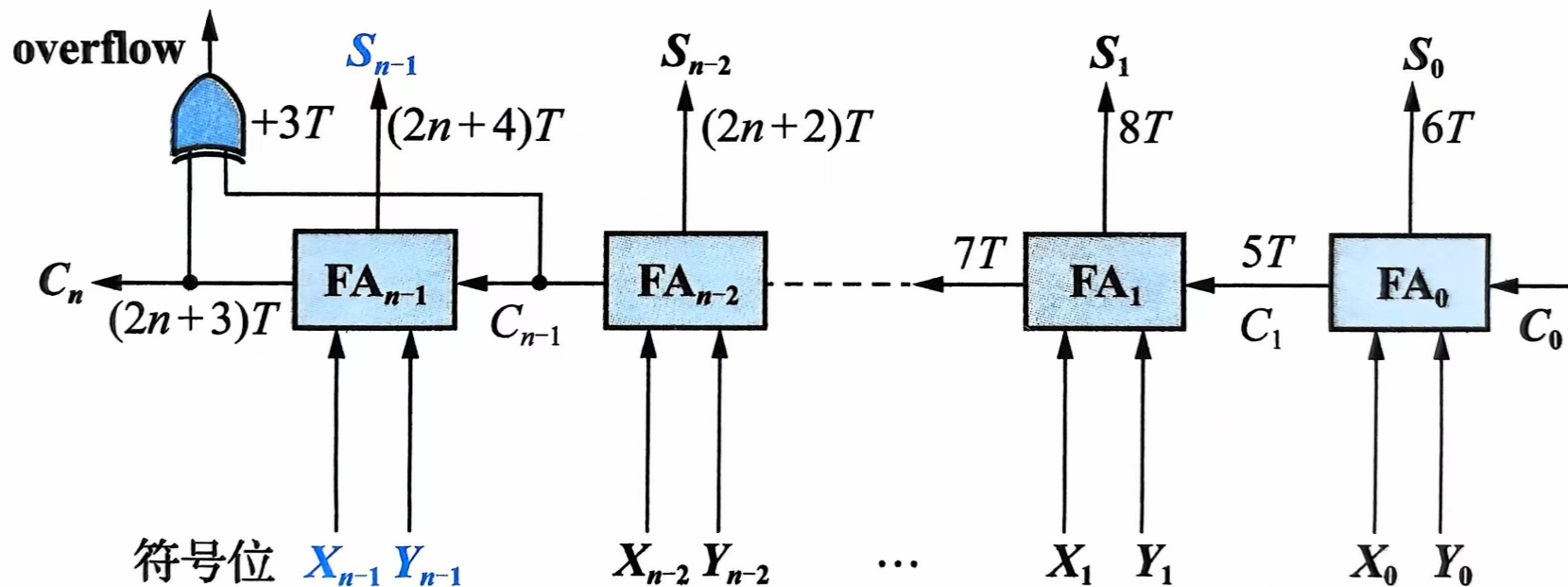
全加器逻辑框图



二. 定点数加减运算

(2) 多位串行加法器(Ripple Carry Adder)

将 n 个全加器的进位串联即可得到 n 位串行加法器，也称行波进位加法器。



多位串行加法器逻辑框图

既可以用于有符号数运算，也可以用于无符号数运算，但二者在溢出检测上有区别。

- 对于无符号数的加法运算，溢出检测信号就是 C_n ；
- 对于有符号数的溢出检测信号overflow

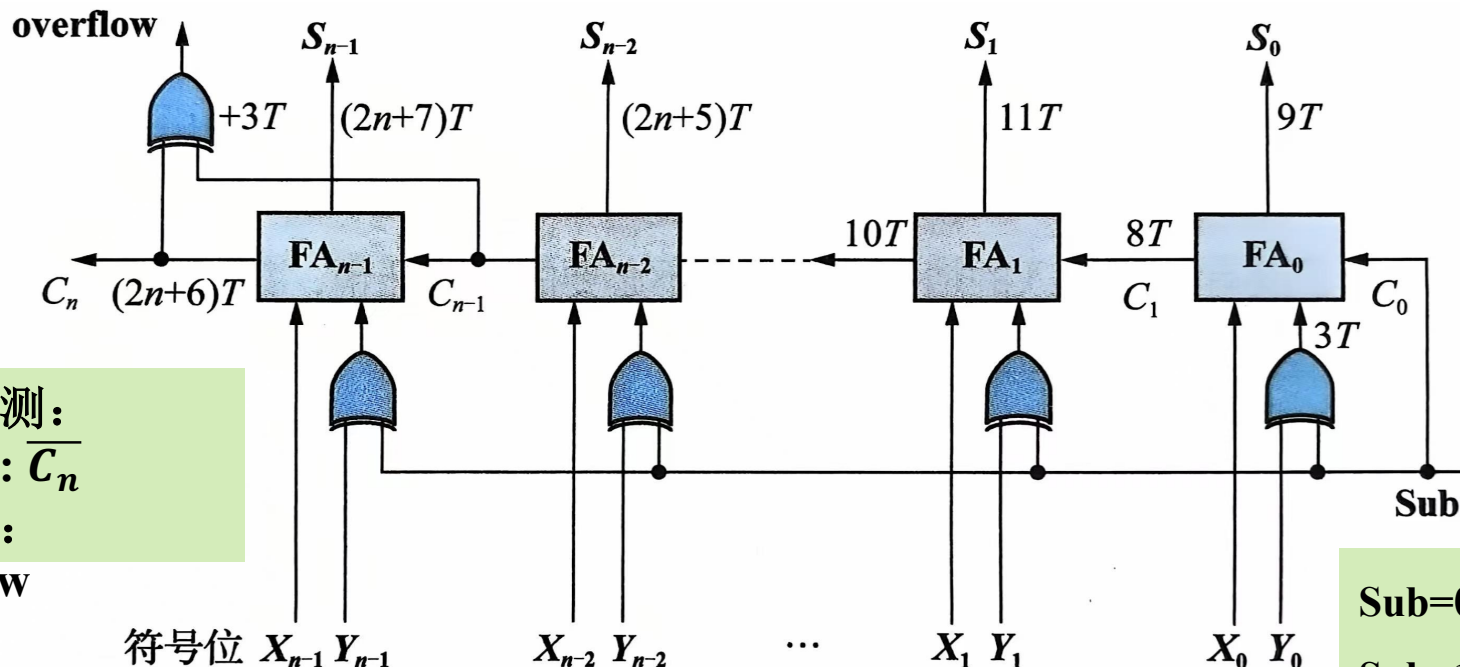


二. 定点数加减运算

(3) 可控加减法器(controlled Adder/Subtractor)

$$[X]_{\text{补}} - [Y]_{\text{补}} = [X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$

减法的关键是求 $[-Y]_{\text{补}}$ ，即对 $[Y]_{\text{补}}$ 连同符号逐位取反，末尾加1。



溢出检测：
无符号： $\overline{C_n}$
有符号：
overflow

Sub=0, 送入Y
Sub=1, 送入Y反码

多位可控加减法电路逻辑框图

Sub连接到加法器最低位的进位输入，实现了对Y操作数逐位取反、末位加1的求补过程，从而完成减法操作。



二. 定点数加减运算

(4) 先行进位加法器 (Carry Look-ahead Adder)

n位串行加法电路中，和数与进位输出的逻辑表达式为：

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = (X_i \cdot Y_i) + (X_i \oplus Y_i) \cdot C_i$$

若令 $G_i = X_i \cdot Y_i$, $P_i = X_i \oplus Y_i$ 。

- 当 $G_i=1$ ， C_{i+1} 一定为1。故将 G_i 称为进位产生函数。
- 当 $P_i=1$ ，也就是 X_i 、 Y_i 相异（有一个为1）时，进位输入信号 C_i 才能传递到进位输出 C_{i+1} 处，故将 P_i 称为进位传递函数。
- 因此有：

$$S_i = P_i \oplus C_i \quad (3-7a)$$

$$C_{i+1} = G_i + P_i \cdot C_i \quad (3-7b)$$



二. 定点数加减运算

① 先行进位 (CLA) 方式

$$\begin{aligned} G_i &= X_i \cdot Y_i \\ P_i &= X_i \oplus Y_i \end{aligned}$$

由 $C_{i+1} = G_i + P_i \cdot C_i$ ，可写出：

$$C_1 = G_0 + P_0 C_0 \quad (3-8a)$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0 \quad (3-8b)$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \quad (3-8c)$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \quad (3-8d)$$

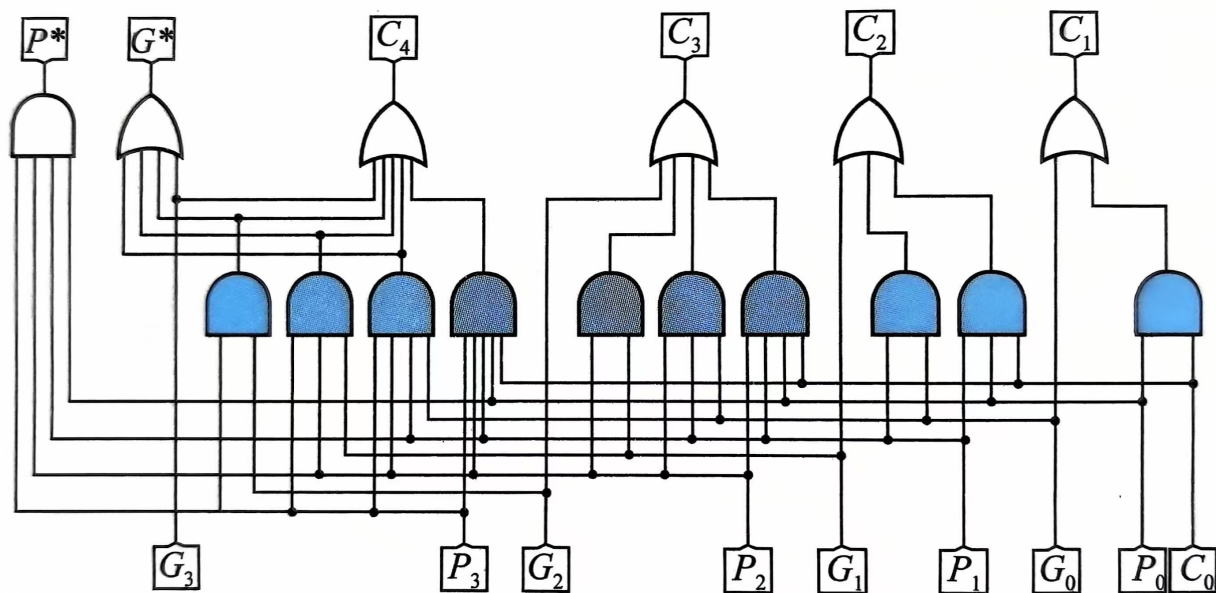
.....

$$C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \dots + P_{n-1} P_{n-2} \dots P_1 P_0 C_0 \quad (3-8e)$$

- 高位进位输出可以由已知的变量 G_i 、 P_i 以及 C_0 经过逻辑运算得到。
- 根据公式利用额外的组合逻辑电路提前产生各位加法运算需要的所有进位输入，再利用 $S_i = P_i \oplus C_i$ 进行一级异或门运算即可得到最终的和数，这就是先行进位的基本原理。



4位先行进位（CLA）电路：



可级联的4位先行进位电路

- 图中与门电路的最大引脚数为5，先行进位电路的位数越多，扇入系数越大，制造难度越大，这也是最终选择4位一组进行先行进位的原因。假设2~5输入引脚的逻辑门的时间延迟相同，则该电路的总时间延迟为2T。
- $G^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$ 称为成组进位生成函数
- $P^* = P_3P_2P_1P_0$ 称为成组进位传递函数



二. 定点数加减运算

4位先行进位（CLA）电路：

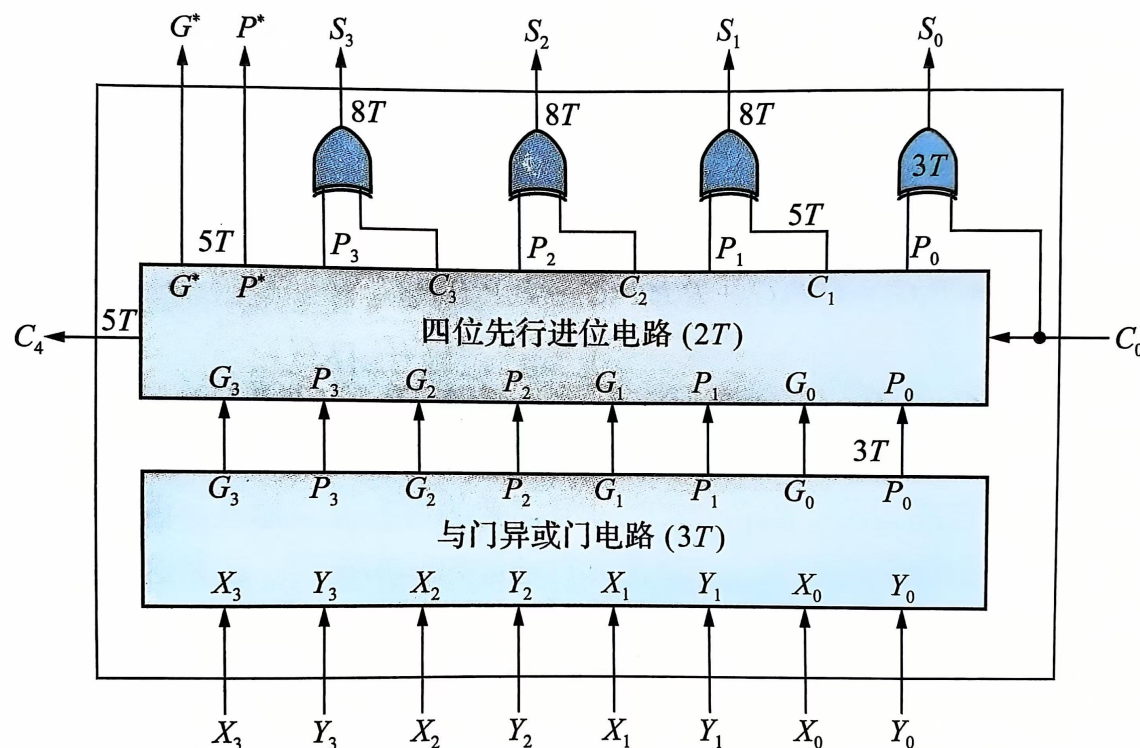
- 定义了成组进位生成函数 G^* 和成组进位传递函数 P^* ，则有 $C_4 = G^* + P^*C_0$ 和 $C_1 = G_0 + P_0C_0$ 形式完全相同，也就是4位一组的进位信号可以采用类似的原理进行成组的先行进位。
- 常见的4位快速先行进位集成电路有SN74182芯片，有兴趣的同学可以查阅相关芯片手册。



二. 定点数加减运算

② 并行加法器

- 4位先行进位电路，加上生成 G^* 和的与门异或门电路，再加上4个异或门就可以构成4位快速加法器。



4位快速加法器

4位全加器的时间延迟为
 $8T=3T+2T+3T$ ，相比4位
串行加法器 $(2n+4)T=12T$
，其性能提升1.5倍。

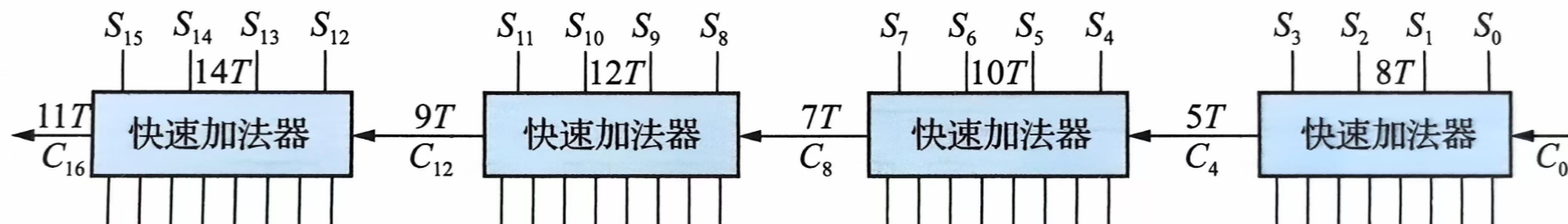
4位快速加法器集成电路
有SN74181芯片，该芯片
支持16种4位运算和成组
先行进位函数，有兴趣
的读者可以自行查阅相
关芯片手册。



二. 定点数加减运算

有了4位快速加法器，如何构建更大位宽的加法器电路？

例如16位加法器，最简单的方法是将4个快速加法器的进位链进行串联，如图所示。



16位组内并行、组间串行加法器

由于所有快速加法器中的与门异或门电路都可以并行运行，图中 C_4 、 C_8 、 C_{12} 、 C_{16} 的时间延迟分别为 $5T$ 、 $7T$ 、 $9T$ 、 $11T$ ，4组和数还需要再加一个异或门时间延迟 $3T$ ， $S_0 \sim S_3$ 、 $S_4 \sim S_7$ 、 $S_8 \sim S_{11}$ 、 $S_{12} \sim S_{15}$ 的时间延迟为 $8T$ 、 $10T$ 、 $12T$ 、 $14T$ ，因此电路的关键时间延迟为 $14T$ 。

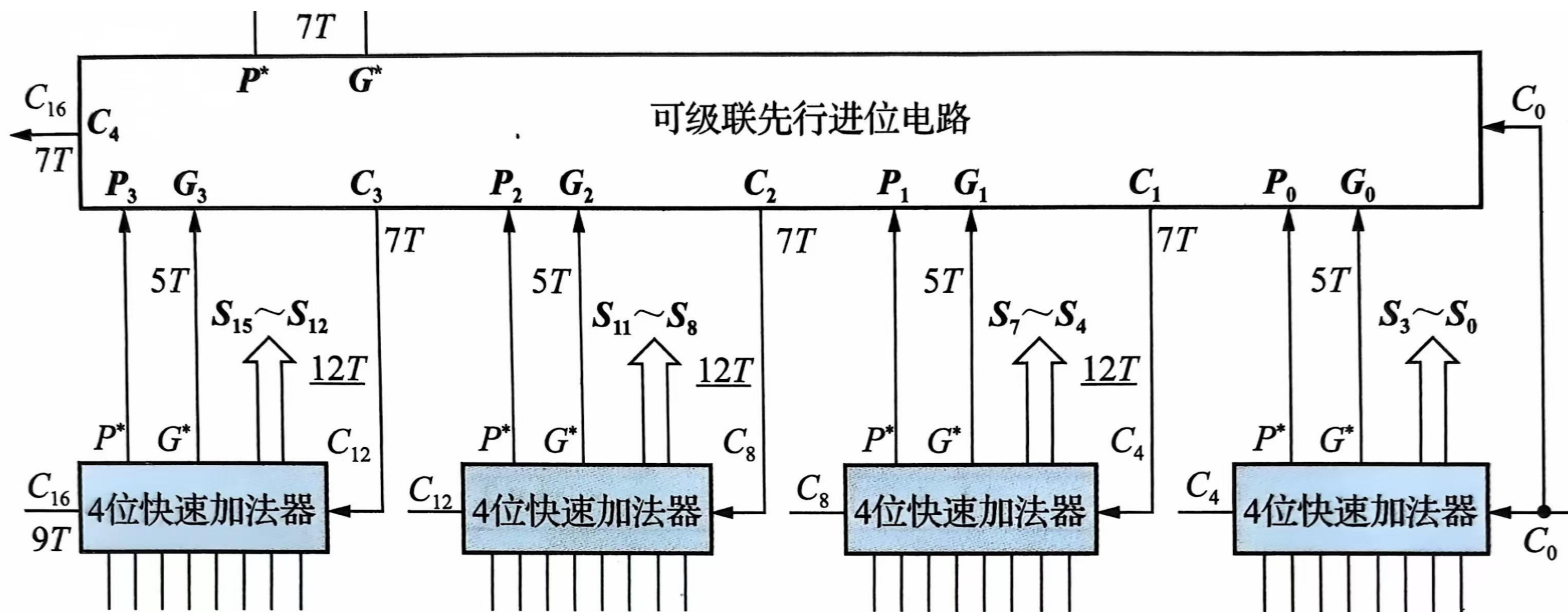
相比传统串行的加法器的 $(2n+4)T = 36T$ ，其性能提升了约2.6倍。



二. 定点数加减运算

有无方法进一步提升其性能呢？

将4个4位快速加法器输出的成组进位生成、传递函数 G^* 和 P^* 及 C_0 连接到先行进位电路的输入端，即可先行产生 C_4 、 C_8 、 C_{12} 、 C_{16} 4个进位信号。再将对应信号连接到相应的快速加法器的进位输入端即可构成16位组内并行进位、组间并行进位的快速加法器。



16位组内并行、组间并行加法器

整个电路的关键时间延迟为和数信号的时间延迟 $12T$ 。



4. 原码加/减运算*

- 用于IEEE754标准的浮点数尾数运算
- 符号位和数值部分分开处理
- 仅对数值部分进行加减运算，符号位起判断和控制作用
- 规则如下：
 - 比较两数符号，对加法实行“同号求和，异号求差”，对减法实行“异号求和，同号求差”。
 - 求和：数值位相加，和的符号取被加数（被减数）的符号。若最高位产生进位，则结果溢出。
 - 求差：被加数（被减数）加上加数（减数）的求补。
 - a) 最高数值位产生进位表明加法结果为正，所得数值位正确。差的符号位取被加数（被减数）的符号；
 - b) 最高数值位没产生进位表明加法结果为负，得到的是数值位的补码形式，需对结果求补，还原为绝对值形式的数值位。符号位为被加数（被减数）的符号取反。



二. 定点数加减运算

例1: 已知 $[X]_{\text{原}} = 1.0011$, $[Y]_{\text{原}} = 1.1010$, 要求计算 $[X+Y]_{\text{原}}$

解: 由原码加减运算规则知: 同号相加, 则求和, 和的符号同被加数符号

。

所以: 和的数值位为: $0011 + 1010 = 1101$ (ALU中无符号数相加)

和的符号位为: 1 求和: 直接加, 有进位则溢出, 符号同被

$$[X+Y]_{\text{原}} = 1.1101$$

例2: 已知 $[X]_{\text{原}} = 1.0011$, $[Y]_{\text{原}} = 1.1010$, 要求计算 $[X-Y]_{\text{原}}$

解: 由原码加减运算规则知: 同号相减, 则求差 (补码减法)

差的数值位为: $0011 + (1010)_{\text{补}} = 0011 + 0110 = 1001$

最高数值位没有产生进位, 表明加法结果为负, 需对1001求补, 还原为绝对值形式的数值位。即: $(1001)_{\text{补}} = 0111$

差的符号位为 $[X]_{\text{原}}$ 的符号位取反, 即: 0

求差: 加补码, 不会溢出, 符号分情况

$$[X-Y]_{\text{原}} = 0.0111$$



二. 定点数加减运算

5. 移码加减运算*

标准移码: $[X]_{\text{移}} = 2^n + X$

移码: 将补码的符号位取反即可

定点整数移码的加减运算法则:

- ① 对两移码求和差;
- ② 对结果进行修正, 即将结果的符号取反。

【证明】 设X、Y为整数, 机器字长n+1位

$$[X + Y]_{\text{移}} = 2^n + X + Y = 2^n + ([X]_{\text{移}} - 2^n) + ([Y]_{\text{移}} - 2^n)$$

- 2ⁿ: 符号取反

$$= [X]_{\text{移}} + [Y]_{\text{移}} - 2^n$$

$$[X - Y]_{\text{移}} = [X]_{\text{移}} + [-Y]_{\text{移}} - 2^n$$

$$[-Y]_{\text{移}} = 2^n + (-Y) = 2^n + (2^n - [Y]_{\text{移}}) = 2^{n+1} - [Y]_{\text{移}} = ((2^{n+1} - 1) - [Y]_{\text{移}}) + 1$$

= $[Y]_{\text{移}}$ 求补

[Y]_移 按位取反 末位加1



二. 定点数加减运算

【例3.9】用8位移码表示十进制数的57和-35。并且用移码运算求两数和与差的移码。

解：十进制数的57和-35用移码表示如下：

$$[57]_{\text{移}} = 1011 \ 1001 \quad [-35]_{\text{移}} = 0101 \ 1101$$

• 求两者之和：

①首先求 $[57]_{\text{移}} + [-35]_{\text{移}}$

$$= 1011 \ 1001 + 0101 \ 1101$$

$$= 0001 \ 0110;$$

②再将符号位取反，从而得到

$$[57 + (-35)]_{\text{移}} = 1001 \ 0110$$


$$\begin{array}{r} 1011 \ 1001 \\ + 0101 \ 1101 \\ \hline 1 \ 0001 \ 0110 \end{array}$$



二. 定点数加减运算

$$[57]_{\text{移}} = 1011\ 1001$$

$$[-35]_{\text{移}} = 0101\ 1101$$

- 求两者之**差**:

①首先求 $[57]_{\text{移}} - [-35]_{\text{移}}$

$$= [57]_{\text{移}} + [[-35]_{\text{移}}]_{\text{求补}}$$

$$= 1011\ 1001 + 1010\ 0011$$

$$= 0101\ 1100$$

- ②再将符号位取反, 从而得到

$$[57 - (-35)]_{\text{移}} = \mathbf{1}101\ 1100$$

- 问题:** 移码加减运算器怎样实现?



$$\begin{array}{r} 1011\ 1001 \\ + 1010\ 0011 \\ \hline \mathbf{1}0101\ 1100 \end{array}$$

将n位加减法器结果输出端的最高位（即符号位）加一个反相器，即可构成移码加减法运算器。



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器
2. 浮点运算器



三. 定点数乘除运算

从计算机硬件角度看，实现乘法运算的方法主要有以下两种。

(1) 利用多位加法器循环累加实现乘法运算，这种方法硬件开销小，但必须采用时序电路进行控制，需多个时钟周期才能得到运算结果。

(2) 采用加法器阵列构成的纯组合逻辑电路实现乘法运算，这种方法只需要一个时钟周期即可完成运算，但硬件开销较大。



三. 定点数乘除运算

1. 原码一位乘法

以小数为例

(1) 乘积符号的确定

$$\text{设}[x]_{\text{原}} = x_0 \cdot x_1 x_2 \cdots x_n$$

$$[y]_{\text{原}} = y_0 \cdot y_1 y_2 \cdots y_n$$

设乘积为P，符号位为 P_f 。同号相乘为正，异号相乘为负。

因此，可得：

$$P_f = x_0 \oplus y_0$$

(2) 乘积的数值

$$|P| = |x| \times |y|$$

原码表示法
：符号+绝对
值



三. 定点数乘除运算

手动计算二进制数乘法的过程:

$$x = 0.1101 \quad y = 0.1011$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline \end{array}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline \end{array}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \end{array} \quad y_4 \times |x| \times 2^{-4}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \end{array} \quad y_3 \times |x| \times 2^{-3}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 0000 \end{array} \quad y_2 \times |x| \times 2^{-2}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \end{array} \quad y_1 \times |x| \times 2^{-1}$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 0.10001111 \end{array}$$

从计算过程可知，手动运算的主要思路就是将乘数 y 的每一位按位权乘上被乘数 $|x|$ 得到位积 $y_i \times |x| \times 2^{-i}$ ，然后将 n 个位积累加求和，即：

$$|P| = \sum_{i=1}^n y_i \times |x| \times 2^{-i}$$

(3-9)



三. 定点数乘除运算

$$\begin{aligned} |P| &= \sum_{i=1}^n y_i \times |x| \times 2^{-i} = 2^{-1}y_1|x| + 2^{-2}y_2|x| + \cdots + 2^{-n}y_n|x| \\ &= (((\underbrace{(0 + y_n|x|)}_{P_1})2^{-1} + y_{n-1}|x|) \underbrace{2^{-1}}_{P_2} + y_{n-2}|x|) \underbrace{2^{-1}}_{P_3} + \cdots + y_1|x| \underbrace{2^{-1}}_{P_n} \end{aligned}$$

该式演化为如下的递推公式：

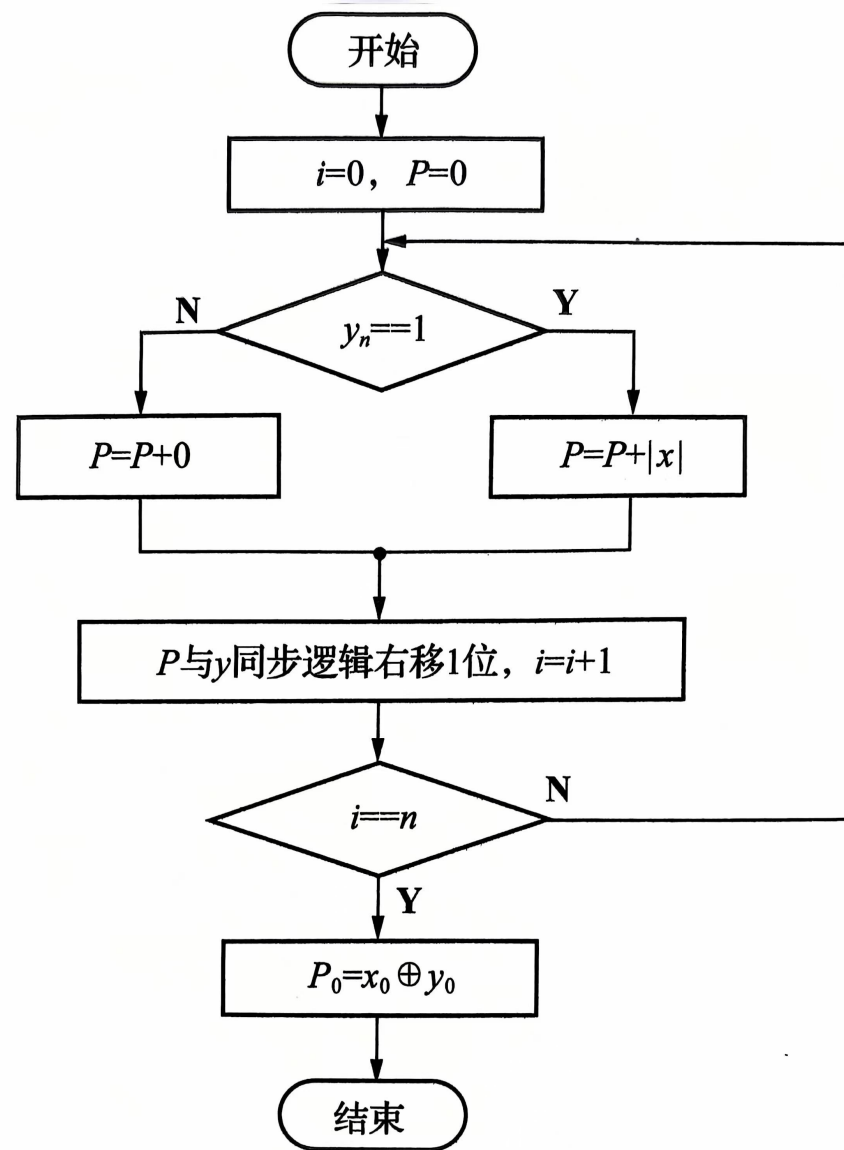
$$P_{i+1} = (P_i + y_{n-i}|x|)2^{-1}, \quad i = 0, 1, 2, \dots, n-1 \quad (3-10)$$

- 可以采用递归的方式计算乘积，设置一个累加寄存器存放部分积P，初始值 $P_0=0$ ，每得到一个位积就将位积累加到累加寄存器中，然后逻辑右移一位得到新的P值。



三. 定点数乘除运算

- 与手工计算相比，以多次累加代替所有部分积同时相加，用部分积右移操作代替手动计算中位积的左移操作，这不仅使部分积的相加运算始终在固定位置上进行，还可以将 $2n$ 位长度的加法器变成 n 位长度加法器，有效减少硬件开销。



原码一位乘法算法流程图



小结

- 原码一位乘法运算可用循环累加和逻辑右移实现。

如：乘数位数 $n = 4$ ，累加4次，右移4次

- 由乘数的末位决定被乘数是否与原部分积相加，然后右移1位形成新的部分积，同时乘数右移1位（末位移丢），空出高位存放部分积的低位。
- 被乘数只与部分积的高位相加。

【例3.10】 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$

解： 数值部分的运算

部分积		乘数	说明
0 . 0 0 0 0		1 1 0 <u>1</u>	部分积 初态 $P_0 = 0$
+ 0 . 1 1 1 0			$+ x^*$
逻辑右移	0 . 1 1 1 0 0 . 0 1 1 1	0 1 1 <u>0</u>	$\rightarrow 1$, 得 P_1
	+ 0 . 0 0 0 0		$+ 0$
逻辑右移	0 . 0 1 1 1 0 . 0 0 1 1	0 1 0 1 <u>1</u>	$\rightarrow 1$, 得 P_2
	+ 0 . 1 1 1 0		$+ x^*$
逻辑右移	1 . 0 0 0 1 0 . 1 0 0 0	1 0 1 1 0 <u>1</u>	$\rightarrow 1$, 得 P_3
	+ 0 . 1 1 1 0		$+ x^*$
逻辑右移	1 . 0 1 1 0 0 . 1 0 1 1	1 1 0 0 1 1 0	$\rightarrow 1$, 得 P_4

【例3.10】 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$

因此，计算的结果为：

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$|x| \cdot |y| = 0.10110110$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.10110110$$

原码一位乘法特点：

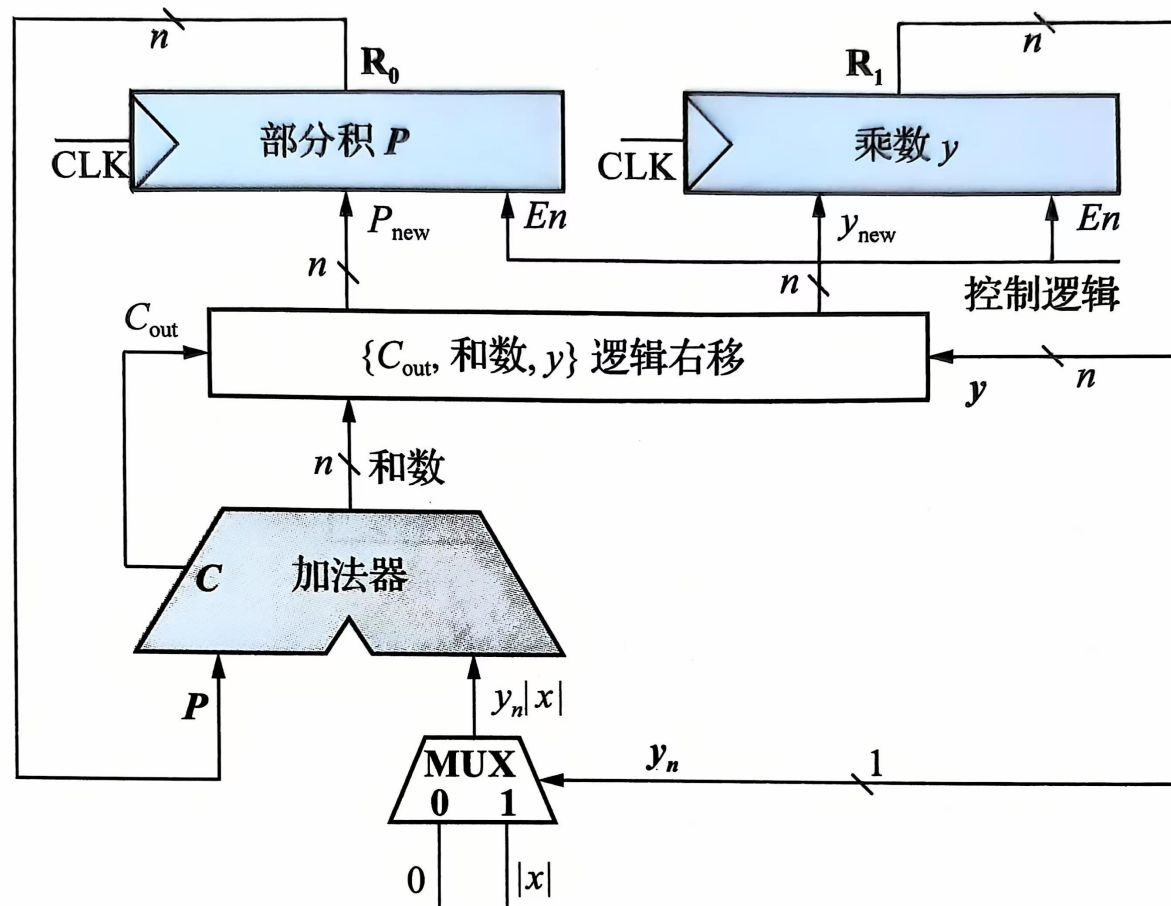
绝对值运算

用移位的次数判断乘法是否结束

逻辑移位



(3) 原码一位乘法器的实现



原码一位乘法的硬件逻辑

寄存器是时序逻辑，需要连接时钟信号，当使能控制端 $En=1$ ，时钟触发时输入端数据会载入寄存器。

控制逻辑受时钟驱动，负责循环计数和算术移位结果的载入控制，加法器和算术移位逻辑是组合逻辑，会自动进行累加与逻辑右移操作。但运算结果必须由控制逻辑控制使能端并配合时钟才能载入寄存器 R_0 、 R_1 中，控制逻辑必须保证前 n 个时钟 $En=1$ ，第 $n+1$ 个时钟之后 $En=0$ ，这样 $2n$ 位乘积会最终锁存在 R_0 、 R_1 这两个 n 位寄存器中。



2. 补码一位乘法

(1) 布斯 (Booth) 算法的推导

设 $[x]_{\text{补}} = x_0.x_1x_2 \dots x_n$, $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$ 。

① 设被乘数 x 符号任意, 乘数 y 符号为正, 则

$$[x]_{\text{补}} = x_0.x_1x_2 \dots x_n, [y]_{\text{补}} = 0.y_1y_2 \dots y_n$$

根据补码定义, $[x]_{\text{补}} = 2 + x = 2^{n+1} + x \pmod{2}$, $[y]_{\text{补}} = y$

$$\begin{aligned} [x]_{\text{补}} \times [y]_{\text{补}} &= (2^{n+1} + x) \times y = 2^{n+1} \times y + x \times y \\ &= 2 \times y_1y_2 \dots y_n + x \times y \end{aligned}$$

注意: 式中 $y_1y_2 \dots y_n$ 是一个整数, 根据模 2 的运算性质,

$2 \times y_1y_2 \dots y_n = 2$, 因此:

$$[x]_{\text{补}} \times [y]_{\text{补}} = 2 + x \times y = [x \times y]_{\text{补}}$$

$$[x \times y]_{\text{补}} = [x]_{\text{补}} \times y \quad (3-11)$$

② 设被乘数 x 符号任意，乘数 y 符号为负，则根据补码定义：

$$[x]_{\text{补}} = x_0.x_1x_2 \dots x_n, \quad [y]_{\text{补}} = 1.y_1y_2 \dots y_n = 2 + y$$

所以，

$$y = [y]_{\text{补}} - 2 = 1.y_1y_2 \dots y_n - 2 = 0.y_1y_2 \dots y_n - 1$$

$$x \times y = x \times (0.y_1y_2 \dots y_n - 1) = x \times 0.y_1y_2 \dots y_n - x \quad (3-12)$$

对（3-12）两边同时求补，并利用补码减法公式展开等式右项可得：

$$[x \times y]_{\text{补}} = [x \times 0.y_1y_2 \dots y_n]_{\text{补}} - [x]_{\text{补}} \quad (3-13)$$

根据（3-11）可得：

$$[x \times y]_{\text{补}} = [x]_{\text{补}} \times 0.y_1y_2 \dots y_n - [x]_{\text{补}} \quad (3-14)$$

将式（3-11）和（3-14）综合起来，引入 y_0 位，即可得到补码一位乘法的统一算式：

$$[x \times y]_{\text{补}} = [x]_{\text{补}} \times 0.y_1y_2 \dots y_n - [x]_{\text{补}} \times y_0 \quad (3-15)$$



$[x \cdot y]_{\text{补}}$

$$=[x]_{\text{补}} \cdot (y_1 \cdot 2^{-1} + y_2 \cdot 2^{-2} + \dots + y_n \cdot 2^{-n}) - [x]_{\text{补}} \cdot y_0 \quad \begin{matrix} 2^{-1} = 2^0 - 2^{-1} \\ 2^{-2} = 2^{-1} - 2^0 \end{matrix}$$

$$=[x]_{\text{补}} \cdot (-y_0 \cdot 2^0 + y_1 \cdot 2^{-1} + y_2 \cdot 2^{-2} + \dots + y_n \cdot 2^{-n})$$

$$=[x]_{\text{补}} \cdot [-y_0 \cdot 2^0 + (y_1 \cdot 2^0 - y_1 \cdot 2^{-1}) + (y_2 \cdot 2^{-1} - y_2 \cdot 2^{-2}) + \dots + (y_n \cdot 2^{-(n-1)} - y_n \cdot 2^{-n})]$$

$$=[x]_{\text{补}} \cdot [(y_1 - y_0) \cdot 2^0 + (y_2 - y_1) \cdot 2^{-1} + \dots + (y_n - y_{n-1}) \cdot 2^{-(n-1)} + (0 - y_n) \cdot 2^{-n}]$$

$$= ((0 + (0 - y_n)[x]_{\text{补}}) 2^{-1} + (y_n - y_{n-1})[x]_{\text{补}}) 2^{-1} + \dots + (y_2 - y_1)[x]_{\text{补}} 2^{-1} + (y_1 - y_0)[x]_{\text{补}}$$

该公式同样可以演变成如下的递推公式：

$$P_{i+1} = (P_i + (y_{n-i+1} - y_{n-i}) [x]_{\text{补}}) 2^{-1} \quad (3-16)$$

$$i=0, 1, 2, \dots, n-1, \quad y_{n+1}=0, \quad P_0=0$$



三、定点数乘除运算

根据（3-16），有如下式子：

$$P_1 = (P_0 + (y_{n+1} - y_n)[x]_{\text{补}}) 2^{-1}$$

$$P_2 = (P_1 + (y_n - y_{n-1})[x]_{\text{补}}) 2^{-1}$$

.....

$$P_n = (P_{n-1} + (y_2 - y_1)[x]_{\text{补}}) 2^{-1}$$

$$[x \cdot y]_{\text{补}} = P_{n+1} = P_n + (y_1 - y_0)[x]_{\text{补}}$$

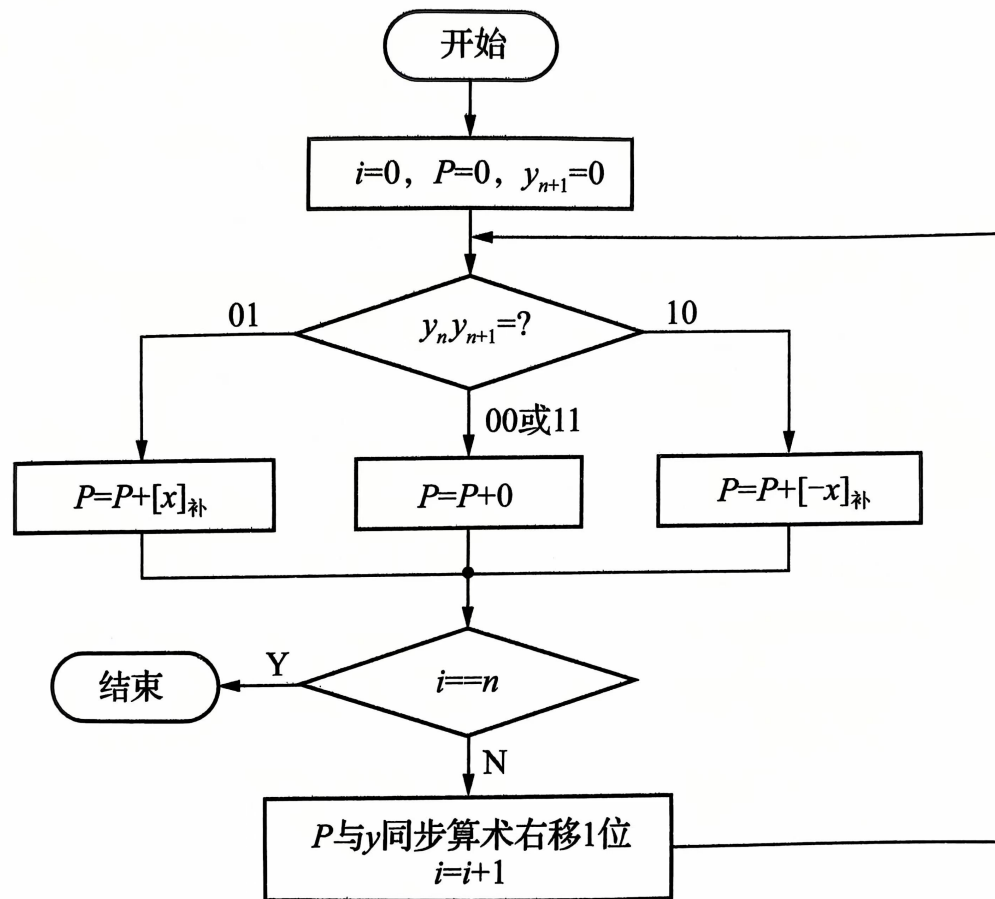
这里 $y_{n+1}=0$ ，人为设置的附加位。

- 这与原码一位乘法的递推公式 $P_{i+1} = (P_i + y_{n-i}|x|)2^{-1}$ 非常类似，只是这里每次累加的值都不一样。
- 另外，补码运算需要 $n+1$ 次累加， n 次移位。



(2) 补码一位乘法的算法

归纳上面推导的结果，便可以得到补码一位乘法算法，如图所示。



补码一位乘法算法流程图



(2) 补码一位乘法的算法

1) 乘数采用单符号位，末位增设附加位 y_{n+1} ，初值为0。

2) 利用 y_{n+1} 与 y_n 的差值判断各步的具体运算

- 差值为1时，累加上 $[x]_{\text{补}}$ ；
- 差值为0时，累加上0；
- 差值为-1时，累加上 $-[x]_{\text{补}} = [-x]_{\text{补}}$ 。

累加完成后需要进行算术右移的操作，初值为0。

3) 按照上述算法进行 $n+1$ 次累加操作， n 次右移操作即可完成乘积运算。

4) 补码乘法中，符号位参与运算，不需要单独计算符号位。

$$[-x]_{\text{补}} = [[x]_{\text{补}}]_{\text{求补}}$$

【例3.11】已知 $x = +0.0011$ $y = -0.1011$ 求 $[x \cdot y]_{\text{补}}$

解:

00.0000	1.0101	0	
+ 11.1101			$+[-x]_{\text{补}}$

补码
右移

11.1101	1 0101		
11.110	1 1010	1	$\rightarrow 1$
+ 00.0011			$+ [x]_{\text{补}}$

补码
右移

00.0001	1 1010		
00.000	1 1101	0	$\rightarrow 1$
+ 11.1101			$+ [-x]_{\text{补}}$

补码
右移

11.1101	1 1101		
11.110	1 1110	1	$\rightarrow 1$
+ 00.0011			$+ [x]_{\text{补}}$

补码
右移

00.0001	1 1110		
00.000	1 1111	0	$\rightarrow 1$
+ 11.1101			$+ [-x]_{\text{补}}$

11.1101 | 1 1 1 1 1 | 最后一步不移位

$$[x]_{\text{补}} = 00.0011$$

$$[y]_{\text{补}} = 1.0101$$

$$[-x]_{\text{补}} = 11.1101$$

$$\therefore [x \cdot y]_{\text{补}} = 1.11011111$$



4) 补码一位乘法的计算核心逻辑:
 $\{P, y\} = \{P + (y_{n+1} - y_n)[x]_{\text{补}}, y\} / 2。$



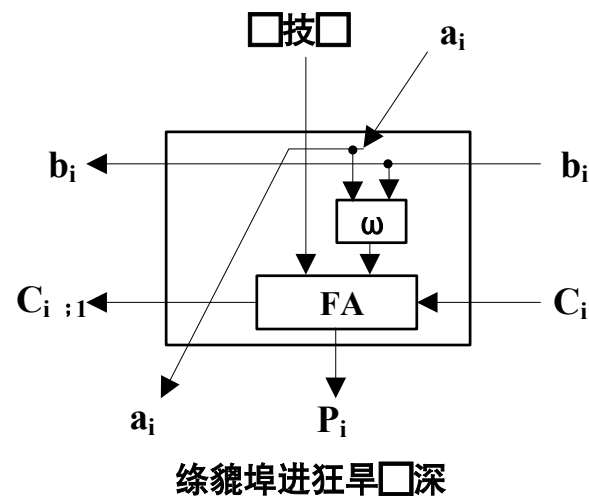
3. 阵列乘法器

- 原码与补码的乘法算法均属于串行乘法。
 - 即：使用同一硬件，通过运算时间上的重复来实现 n 位数据的相乘。
 - 需要的运算时间随着位数的增加线性增长。
- 虽然可以使用两位乘法或者多位乘法以提高运算速度，但是硬件复杂度大大提高。
- 随着大规模集成电路的广泛应用，出现了阵列乘法器，进一步提高了乘法的运算速度。



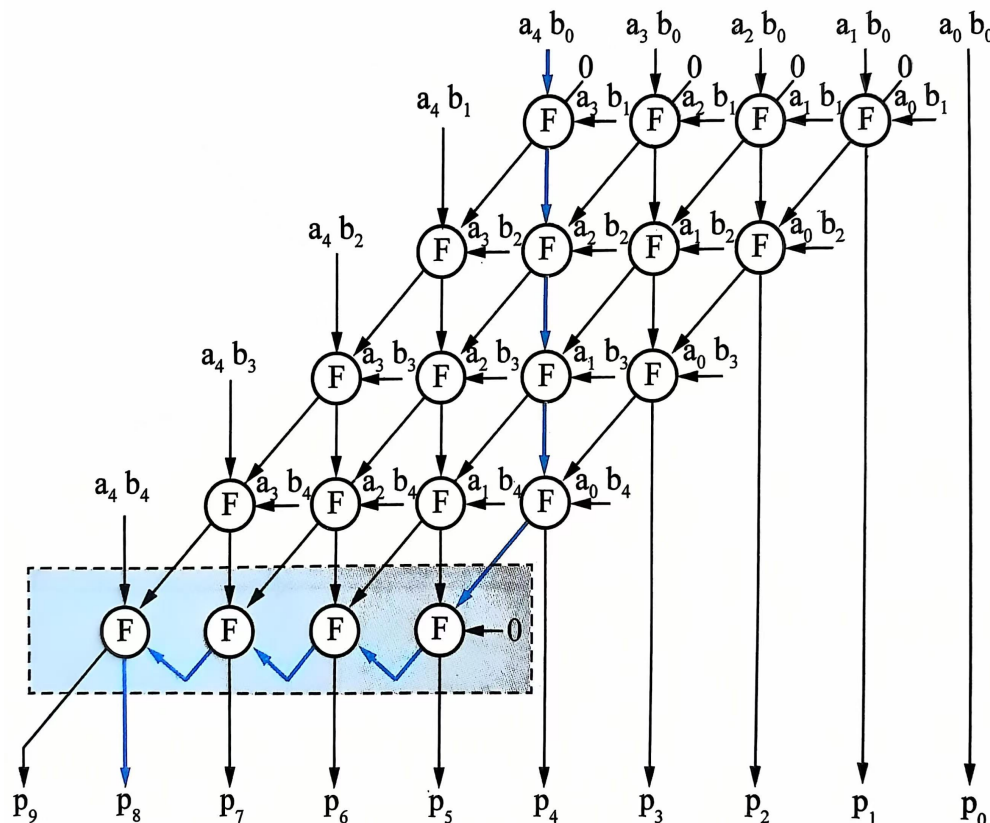
- 阵列乘法器的基本思想：采用类似手动乘法运算的方法，用大量与门阵列同时产生手动乘法中的各乘积项，同时将大量一位全加器按照手动乘法运算的需要构成全加器阵列。

[illegible]





三、定点数乘除运算



一个5位乘5位无符号阵列乘法器的工作原理图

原码一位乘法时间复杂度是 $O(n^2)$

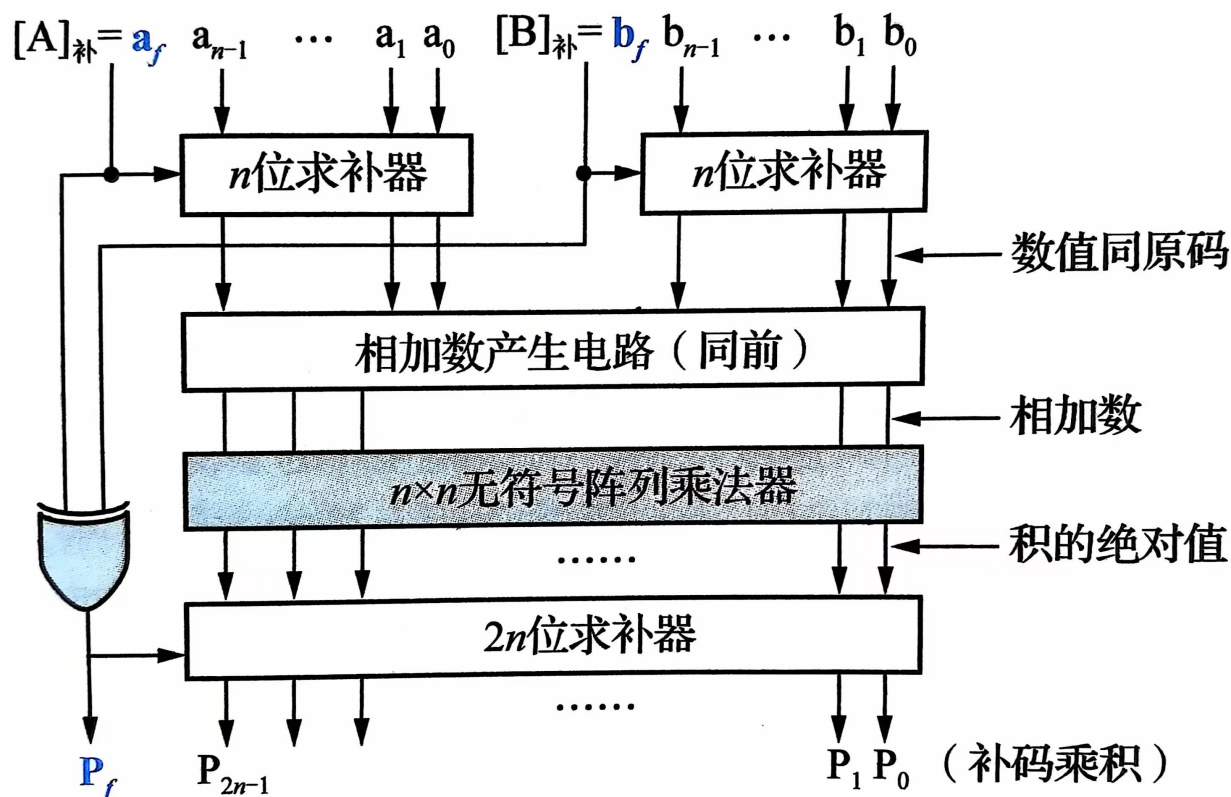
阵列乘法器时间复杂度是 $O(n)$

- 图中阵列乘法器包括 $n(n-1) = 5 \times 4 = 20$ 个全加器FA，全加器的操作数由 $n \times n$ 个与门并行产生，需要 $1T$ 时间延迟。
- 前 $(n-1)=4$ 行内的全加器没有进位依赖关系，行内全加器可并行，时间延迟为 $6T$ 。
- 行与行之间有进位依赖关系，上面的行计算完毕后下面的行才能进行运算。
- 关键延迟路径 P_4 的时间延迟为 $1T + (n-1)6T$ ，而与 P_4 相关的进位输出会早一个时钟周期，其时间延迟为 $1T + (n-1)6T - 1T$ 。
- 最下面一行的全加器是 $n-1=4$ 位串行加法器，串行加法器的时间延迟是 $[2(n-1)+4]T = 12T$ ，所以总时间延迟为 $1T + (n-1)6T - 1T + [2(n-1)+4]T = (8n-4)T = 36T$ 。



补码阵列乘法器

基于无符号阵列乘法器还可以设计原码阵列乘法器和补码阵列乘法器，下图所示是 $n+1$ 位补码阵列乘法器的工作原理。





【例3.9-随堂4】 下列定点数乘法运算的实现方法中，速度最快的是：

A.布斯法（Booth法）

B.原码一位乘法

C.阵列乘法器

D.利用加法和移位指令，通过软件实现



4. 原码除法

以小数为例

被除数 $[x]_{\text{原}} = x_0.x_1x_2\cdots x_n$ 除数 $[y]_{\text{原}} = y_0.y_1y_2\cdots y_n$

商 $[Q]_{\text{原}} = q_0.q_1q_2\cdots q_n$ 余数 $[R]_{\text{原}} = \mathbf{0}.r_1r_2\cdots r_n$

商的符号： $q_0 = x_0 \oplus y_0$

商的数值： $|Q| = |x| \div |y|$

为了保证运算的结果不发生溢出，隐含条件： $|x| < |y|$



手动除法计算的启示

$$\begin{array}{r}
 0.1011 \overline{) 0.1101} \\
 \underline{-0.0101} \downarrow \\
 0.001110 \\
 \underline{-0.0010} \downarrow \\
 0.0000111 \\
 \underline{0.00001011} \\
 0.000001110 \\
 \underline{-0.000001011} \\
 \text{余数 } R \quad 0.000000011
 \end{array}$$

商 Q

R_0

$y \times 2^{-1}$

R_1

$y \times 2^{-2}$

R_2

$y \times 2^{-3}$

R_3

$y \times 2^{-4}$

R_4

操作说明

$x < y, q_0=0$

$R_0 \geq y, q_1=1$, 减除数

$R_1 \geq y, q_2=1$, 减除数

$R_2 < y, q_3=0$, 不减除数

$R_3 \geq y, q_4=1$, 减除数



三、定点数乘除运算

通过分析手动二进制除法的运算过程，可以发现下列规律。

- (1) 除法通过不断上商求余实现。
- (2) 比较余数 R_i 与 $y \cdot 2^{-i}$ （除数右移 i 位）求第 i 位商 q_i ，小于则商上0，否则商上1。
- (3) 如果 $R_i > y \cdot 2^{-i}$ ，进行减法得到新的余数；如果小于余数，则不进行减法。
- (4) 上商后被除数右移一位。
- (5) 继续上商求余直至商的位数满足要求。



三、定点数乘除运算

为便于在计算机上执行除法运算，对手动除法算法进行如下改进

(1) 通过减法运算比较数的大小，并作为上商的依据。

根据差值的符号位判断商值，如差值为负数（符号位为1），不够减，商上0；差值为正数表示够减，商上1，因此，上商位等于差值符号位 r_0 的求非，也就是 $\bar{q} = r_0$ 。

(2) 将除数右移操作改为余数左移操作，并与上商操作统一，使上商能固定在同一个位置上进行。

值得注意的是，每左移一次余数，相当于余数乘2，在求得n位商后，余数R被左移了n次，因此，最后正确的余数应为

$$R \times 2^{-n}$$



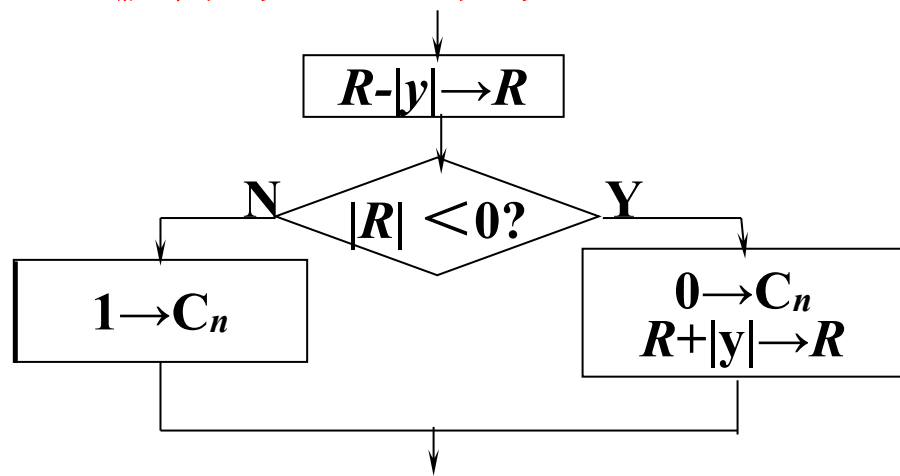
三、定点数乘除运算

原码除法中由于对余数的处理不同，又可分为恢复余数法和
恢复余数法(加减交替法)两种。

(1)恢复余数法

恢复余数法是直接作减法试探，不管被除数（或余数）减除数是否够减，都一律先做减法。

- 若余数为正，表示够减，该位商上“1”；
- 若余数为负，表示不够减，该位商上“0”，并要恢复原来的被除数（或余数）。



商值的确定是通过比较被除数（余数）和除数的绝对值大小，即 $R - |y|$ 实现的，而计算机内只设加法，故将 $R - |y|$ 变为 $R + [-|y|]_{\text{补}}$

【例3.12】 $[x]_{\text{原}} = 1.1001$ $[y]_{\text{原}} = 1.1011$, 求 $[x]_{\text{原}} \div [y]_{\text{原}}$

解: $[y]_{\text{补}} = 0.1011$ $[-y]_{\text{补}} = 1.0101$

	余数 R	商 Q	说明
	00.1001	0.0000	初始余数 $R_0 = x$
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	11.1110	0.000 <u>0</u>	$R_1 < 0$, 商 q_1 上 0, $q = \bar{r}_0$
$+ y $	00.1011		加 $ y $ 恢复余数
	00.1001		
$\leftarrow$	01.0010	0.000	R 、 Q 同步左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	00.0111	0.000 <u>1</u>	$R_2 > 0$, 商 q_2 上 1
$\leftarrow$	00.1110	0.001	左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	00.0011	0.001 <u>1</u>	$R_3 > 0$, 商 q_3 上 1
$\leftarrow$	00.0110	0.011	左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	11.1011	0.011 <u>0</u>	$R_4 < 0$, 商 q_4 上 0
$+ y $	00.1011		加 $ y $ 恢复余数
	00.0110		
$\leftarrow$	00.1100	0.110	左移一位
$+[-y]_{\text{补}}$	11.0101		减 $ y $ 比较
	00.0001	0.110 <u>1</u>	$R_5 > 0$, 商 q_5 上 1

商的符号:

$$x_0 \oplus y_0 = 1$$

故:

$$[x]_{\text{原}} \div [y]_{\text{原}} = 1.1101$$

$$\text{余数 } R = 0.0001 \times 2^{-4}$$

操作次数不固定, 控制电路复杂; 恢复余数降低了执行速度; 因此一般很少采用。



(2) 原码不恢复余数法（原码加减交替法）

• 恢复余数法运算规则

余数 $R_i > 0$ 上商 “1”， $2R_i - |y|$

余数 $R_i < 0$ 上商 “0”， $R_i + |y|$ 恢复余数

$$2(R_i + |y|) - |y| = 2R_i + |y|$$

• 不恢复余数法运算规则

上商 “1” $2R_i - |y|$

上商 “0” $2R_i + |y|$

加减交替

【例3.13】 $[x]_{\text{原}} = 1.1001$ $[y]_{\text{原}} = 1.1011$, 求 $[x]_{\text{原}} \div [y]_{\text{原}}$

解: $[|y|]_{\text{补}} = 0.1011$ $[-|y|]_{\text{补}} = 1.0101$

	C 余数 R	商 Q	说明
	00.1001	0.0000	初始余数 $R_0 = x$
$+[- y]_{\text{补}}$	11.0101		减 $ y $ 比较
	0 11.1110	0.000 0	$R_1 < 0$, 商上 0 , $q = C = \bar{r}_0$
←	11.1100	0.000	R 、 Q 同步左移一位
$+ y $	00.1011		加 $ y $ 比较
	1 00.0111	0.000 1	$R_2 > 0$, 商上 1
←	00.1110	0.00 1	左移一位
$+[- y]_{\text{补}}$	11.0101		减 $ y $ 比较
	1 00.0011	0.00 11	$R_3 > 0$, 商上 1
←	00.0110	0.0 11	左移一位
$+[- y]_{\text{补}}$	11.0101		减 $ y $ 比较
	0 11.1011	0.0 110	$R_4 < 0$, 商上 0
←	11.0110	0. 110	左移一位
$+ y $	00.1011		加 $ y $ 比较
	1 00.0001	0. 1101	$R_5 > 0$, 商上 1

上商位与进位位的值相同

商的符号:

$$x_0 \oplus y_0 = 1$$

故:

$$[x]_{\text{原}} \div [y]_{\text{原}} = 1.1101$$

$$\text{余数 } R = 0.0001 \times 2^{-4}$$

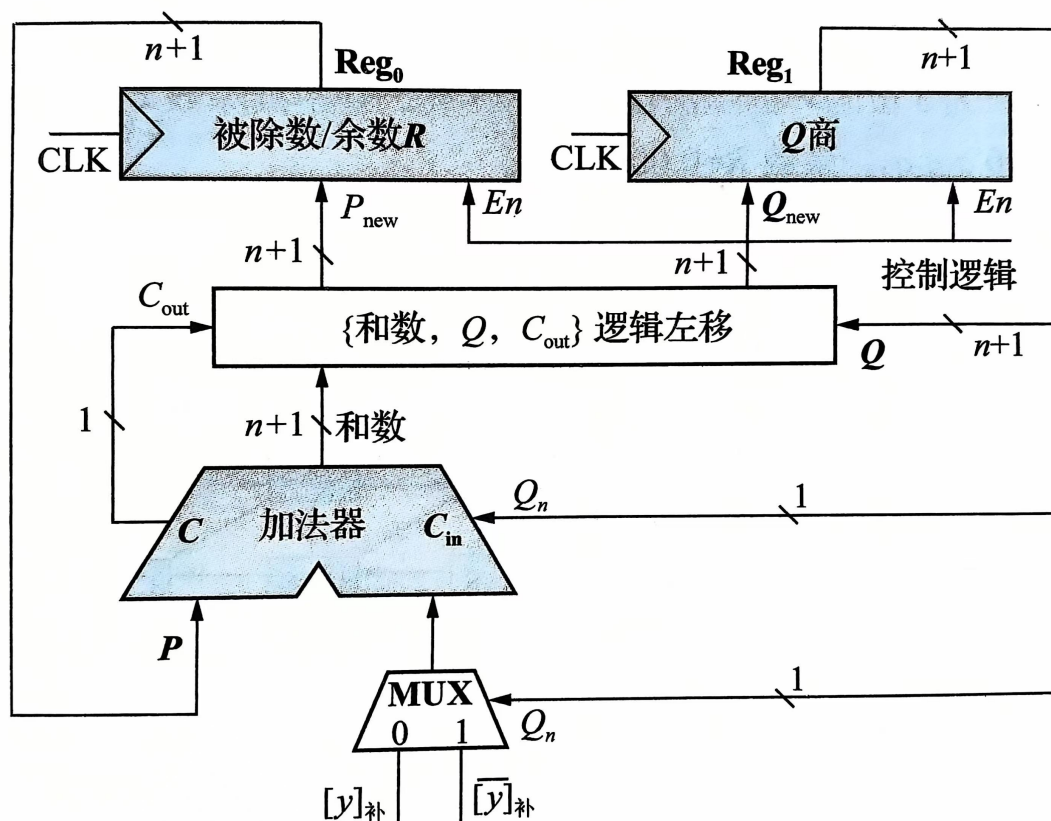


原码加减交替法除法小结

- 符号位不参与运算
- 上商 $n+1$ 次，第一次上商判溢出
- （逻辑）左移 n 次，用移位的次数判断除法是否结束。
- 若最终余数为负，需恢复余数
- 小方框中的数字是运算器进位位的值，可以发现上商位与进位位的值相同， $q = C = \overline{r_0}$ 。因此，在逻辑实现时可用加法器进位位作为上商的控制信号以及可控加减法电路的控制信号。



加减交替法除法器逻辑框图



加法器的运算由商寄存器最低位 Q_n 决定， Q_n 的值是上一步运算所上的商。当 $Q_n=1$ 时，下一步减 $|y|$ ，这里减法采用反码末位加1的方式实现；当 $Q_n=0$ 时，则下一步进行加 $|y|$ 的操作。注意 Q_n 初始值应该为1，以保证第一次运算为减法。

当前的上商位由 C_{out} 决定，{和数、 Q 、 C_{out} }同步左移后，高位部分 P_{new} 被送入余数寄存器 R 中，低位部分 Q_{new} 被送入商寄存器 Q 中，在时钟到来时载入，这样每次新的上商位 C_{out} 都可以加载到商寄存器 Q 的 Q_n 位。



5. 补码除法*

补码除法也分恢复余数法和加减交替法，后者用得较多，在此只讨论加减交替法。

- 补码加减交替法运算规则

- 补码除法的符号位和数值部分是一起参加运算的，因此在算法上不像原码除法那样直观，主要需要解决3个问题：

- ①如何确定商值；
- ②如何形成商符；
- ③如何获得新的余数。



三、定点数乘除运算

①如何确定商值;

- 参加运算的两个数符号任意，当被除数(或部分余数)的绝对值大于或等于除数的绝对值时，称为够减；反之称为不够减。

x 与 y 同号

$$x > 0, y > 0, |x| - |y| = x - y > 0$$

$$x < 0, y < 0, |x| - |y| = -x - (-y) > 0 \Rightarrow x - y < 0$$

x 与 y 异号

$$x > 0, y < 0, |x| - |y| = x - (-y) = (x + y) > 0$$

$$x < 0, y > 0, |x| - |y| = (-x) - y > 0 \Rightarrow x + y < 0$$

$[x]_{\text{补}}$ 和 $[y]_{\text{补}}$	求 $[R_i]_{\text{补}}$	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$
同号	$[x]_{\text{补}} - [y]_{\text{补}}$	同号, “够减”
异号	$[x]_{\text{补}} + [y]_{\text{补}}$	异号, “够减”

① 商值的确定

末位恒置“1”法

$\times.\times\times\times\times$ **1**

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ **同号**
正商

0. $\underbrace{\times\times\times\times}_{\text{原码}}$ **1**

按原码上商 “够减”上“1”
“不够减”上“0”

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ **异号**
负商

1. $\underbrace{\times\times\times\times}_{\text{反码}}$ **1**

按反码上商 “够减”上“0”
“不够减”上“1”

小 结

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$	商	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商 值
同 号	正	够减 (同号)	1
		不够减 (异号)	0
异 号	负	够减 (异号)	0
		不够减 (同号)	1

简 化 为

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同 号	1
异 号	0

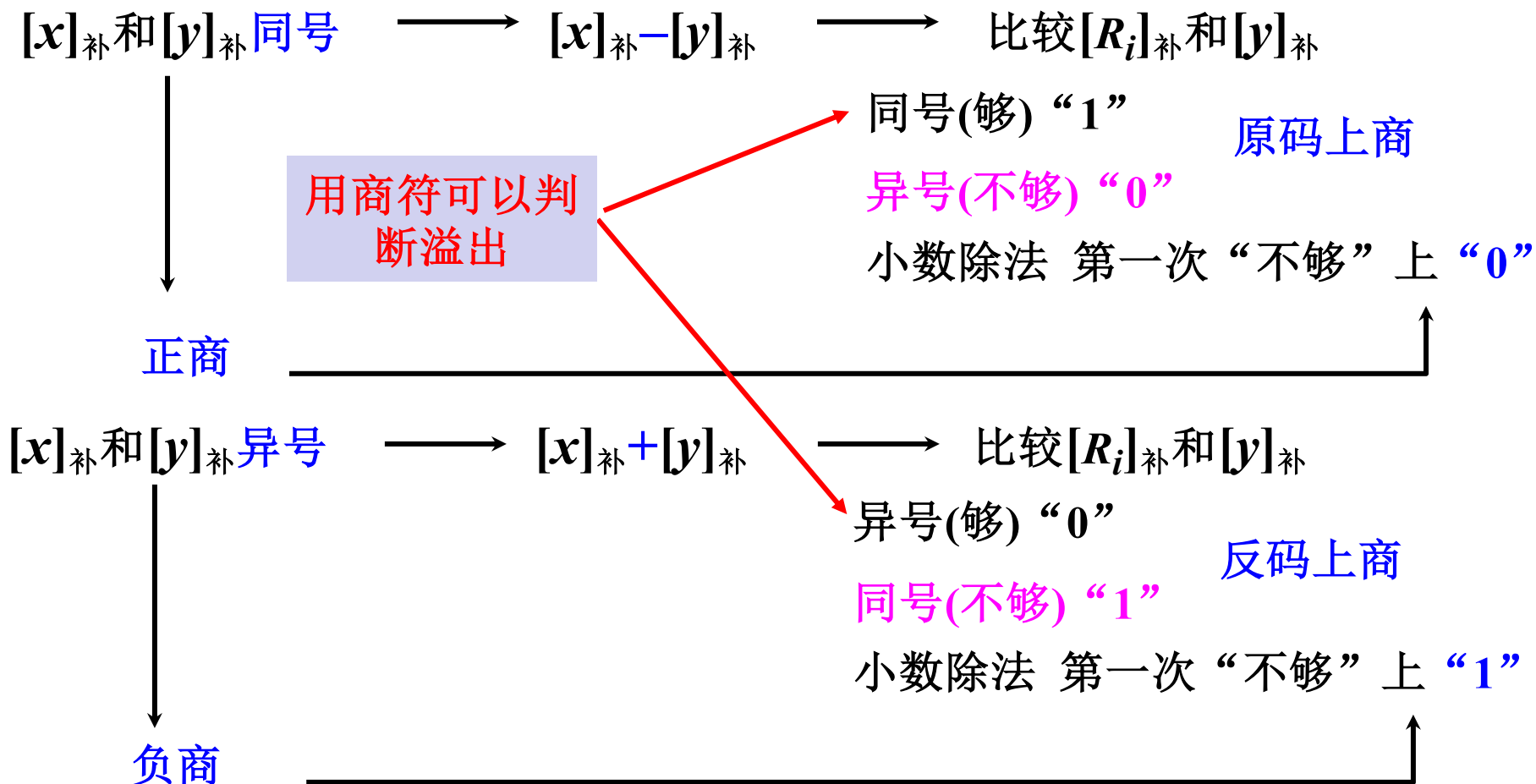


三、定点数乘除运算

② 商符的形成

除法过程中自然形成

在小数定点除法中，被除数的绝对值必须小于除数的绝对值，否则商大于1而溢出。





③ 新余数的获得

加减交替

$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$	商	新余数
同 号	1	$2[R_i]_{\text{补}} + [-y]_{\text{补}}$
异 号	0	$2[R_i]_{\text{补}} + [y]_{\text{补}}$

【例3.14】 设 $x = -0.1011$ $y = 0.1101$ 求 $[\frac{x}{y}]_{\text{补}}$ 并还原成真

解: $[x]_{\text{补}} = 1.0101$ $[y]_{\text{补}} = 0.1101$ $[-y]_{\text{补}} = 1.0011$

被除数(余数)	商	说 明
1.0101 + 0.1101	0.0000	$[x]_{\text{补}}$ 和 $[y]_{\text{补}}$ 异号, $+ [y]_{\text{补}}$
0.0010 0.0100 + 1.0011	0.0001	$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$ 同号上 “1” $\leftarrow 1$ $+ [-y]_{\text{补}}$
1.0111 0.1110 + 0.1101	0.0010	$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$ 异号上 “0” $\leftarrow 1$ $+ [y]_{\text{补}}$
1.1011 1.0110 + 0.1101	0.0100	$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$ 异号上 “0” $\leftarrow 1$ $+ [y]_{\text{补}}$
0.0011 0.0110 + 0.1101	0.1001	$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$ 同号上 “1” $\leftarrow 1$ $+ [-y]_{\text{补}}$
1.0011	1.0011	末位恒置 “1”

求 $[R_i]_{\text{补}}$

$[x]_{\text{补}}$ 和 $[y]_{\text{补}}$ 异号

$[x]_{\text{补}} + [y]_{\text{补}}$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同 号	1
异 号	0

$$\therefore [\frac{x}{y}]_{\text{补}} = 1.0011$$

$$\frac{x}{y} = -0.1101$$

逻辑
左移

逻辑
左移

逻辑
左移



补码除法小结

- 补码除法共上商 $n+1$ 次（末位恒置 1）
第一次为商符
- 第一次商可判溢出
- 加 $n+1$ 次 左移 n 次
- 用移位的次数判断除法是否结束
- 精度误差最大为 2^{-n}

补码除和原码除（加减交替法）比较

	原码除	补码除
商符	$x_0 \oplus y_0$	自然形成
操作	绝对值补码	补码
数上商原则	余数的正负	比较余数和除数的符号
上商次数	$n + 1$	$n + 1$
加法次数	$n + 1$ 或 $n + 2$	$n + 1$
移位	逻辑左移	逻辑左移
移位次数	n	n
第一步操作	$[x]_{\text{补}} - [y]_{\text{补}}$	同号 $[x]_{\text{补}} - [y]_{\text{补}}$ 异号 $[x]_{\text{补}} + [y]_{\text{补}}$



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算

2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器

2. 浮点运算器



1. 浮点加减运算

$$x = M_x \cdot 2^{E_x} \quad y = M_y \cdot 2^{E_y}$$

浮点数加减运算步骤：陀螺

- ① 对阶，使两数的小数点位置对齐。
- ② 尾数求和，将对阶后的两尾数按定点加减运算规则求和(差)。
- ③ 规格化，为增加有效数字的位数，提高运算精度，必须将求和(差)后的尾数规格化。
- ④ 舍入，为提高精度，要考虑尾数右移时丢失的数值位。
- ⑤ 溢出判断，即判断结果是否溢出。



1. 对阶

(1) 求阶差

$$\Delta E = E_x - E_y = \begin{cases} = 0 & E_x = E_y & \text{已对齐} \\ > 0 & E_x > E_y & \begin{array}{l} x \text{ 向 } y \text{ 看齐} \\ y \text{ 向 } x \text{ 看齐} \end{array} & \begin{array}{l} M_x \leftarrow 1, E_x - 1 \\ \checkmark M_x \rightarrow 1, E_y + 1 \end{array} \\ < 0 & E_x < E_y & \begin{array}{l} x \text{ 向 } y \text{ 看齐} \\ y \text{ 向 } x \text{ 看齐} \end{array} & \begin{array}{l} \checkmark M_y \rightarrow 1, E_x + 1 \\ M_y \leftarrow 1, E_y - 1 \end{array} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐

- 小阶对大阶时，右移小阶的尾数舍去的是尾数的低位
 - 大阶对小阶时，左移大阶的尾数舍去的是尾数的高位
- 显然，小阶对大阶更能保证精度。



【例3.15】

$$x = 0.1101 \times 2^{01} \quad y = (-0.1010) \times 2^{11} \quad \text{求 } x+y$$

解: $[x]_{\text{补}} = 00, 01; 00.1101$ $[y]_{\text{补}} = 00, 11; 11.0110$

1. 对阶

① 求阶差 $[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00, 01$

$$\begin{array}{r} + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

阶差为负 (-2) $\therefore M_x \rightarrow 2 \quad E_x + 2$

② 对阶 $[x]'_{\text{补}} = 00, 11; 00.0011$

2. 尾数求和

$$\begin{array}{r} [M_x]'_{\text{补}} = 00.0011 \\ + [M_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \end{array}$$

$\therefore [x+y]_{\text{补}} = 00, 11; 11.1001$

工程上普遍的做法：尾数右移时将最低位的移出位暂时保留，称为保留附加位。其参与中间运算，运算结束后结果规格化后再进行舍入。



3. 规格化

(1) 规格化数的定义

$$R = 2 \quad \frac{1}{2} \leq |M| < 1$$

(2) 规格化数的判断

$M > 0$	规格化形式	$M < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	$0.\mathbf{1} \times \times \dots \times$	原码	$1.\mathbf{1} \times \times \dots \times$
补码	$\mathbf{0.1} \times \times \dots \times$	补码	$\mathbf{1.0} \times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同



3. 规格化

特例

$$M = -\frac{1}{2} = -0.100\dots 0$$

$$[M]_{\text{原}} = 1.100\dots 0$$

$$[M]_{\text{补}} = \boxed{1.1}00\dots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$M = -1$$

$$[M]_{\text{补}} = \boxed{1.0}00\dots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数



(3) 左规

尾数左移一位，阶码减 1，直到数符和第一数位不同为止

上例 $[x+y]_{\text{补}} = 00, 11; 11. 1001$

左规后 $[x+y]_{\text{补}} = 00, 10; 11. 0010$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

(4) 右规

当尾数溢出（>1）时，需右规

即尾数出现 $01. \times \times \dots \times$ 或 $10. \times \times \dots \times$ 时

尾数右移一位，阶码加 1

【例3.16】

$$x = 0.1101 \times 2^{10} \quad y = 0.1011 \times 2^{01}$$

求 $x+y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $[x]_{\text{补}} = 00, 010; 00. 110100$ $[y]_{\text{补}} = 00, 001; 00. 101100$

① 对阶 $[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00, 010 + 11, 111 = 100, 001$

$$\text{阶差为 } +1 \quad \therefore M_y \rightarrow \text{左}, E_y + 1$$

$$\therefore [y]'_{\text{补}} = 00, 010; 00. 010110$$

② 尾数求和

$$[M_x]_{\text{补}} = 00. 110100$$

$$+ [M_y]'_{\text{补}} = 00. 010110$$

$$\hline 01. 001010$$

对阶后的 $[M_y]'_{\text{补}}$

尾数溢出需右规

③ 右规 $[x+y]_{\text{补}} = 00, 010; 01. 001010$

右规后 $[x+y]_{\text{补}} = 00, 011; 00. 100101$

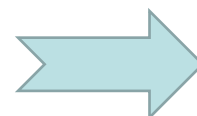
$$\therefore x+y = 0. 100101 \times 2^{11}$$

④ 舍入

在 对阶 和 右规 过程中, 可能出现 尾数末位丢失 引起误差, 需考虑舍入

(1) 0 舍 1 入法

(2) 恒置 “1” 法



【例3.17】 $x = (-\frac{5}{8}) \times 2^{-5}$ $y = (-\frac{7}{8}) \times 2^{-4}$

求 $x-y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $x = (-0.101000) \times 2^{-101}$ $y = (0.111000) \times 2^{-100}$

$[x]_{\text{补}} = 11, 011; 11. 011000$ $[y]_{\text{补}} = 11, 100; 00. 111000$

① 对阶 $[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 11, 011 + 00, 100 = 11, 111$

阶差为 $-1 \quad \therefore M_x \xrightarrow{-1} M_x, E_x+1$

$\therefore [x]_{\text{补}}' = 11, 100; 11. 101100$

② 尾数求和

$[M_x]_{\text{补}}' = 11. 101100$

$+ [-M_y]_{\text{补}} = 11. 001000$

110. 110100

③ 右规

$[x-y]_{\text{补}} = 11, 100; 10. 110100$

右规后 $[x-y]_{\text{补}} = 11, 101; 11. 011010$

$\therefore x-y = (-0.100110) \times 2^{-11} = (-\frac{19}{32}) \times 2^{-3}$



(5) 溢出判断

- 当尾数之和（差）出现 $10.x\ x\ x\ \dots\ x$ 或 $01.x\ x\ x\ \dots\ x$ 时，并不表示溢出，只有将此数右规后，再根据阶码来判断浮点运算结果是否溢出。
- 浮点数的溢出情况由阶码的符号决定，若阶码也用双符号位补码表示，当：
 - $[E_z]_{\text{补}} = 01, xxx\dots x$ ，表示上溢。此时，浮点数真正溢出，机器需停止运算，做溢出中断处理。
 - $[E_z]_{\text{补}} = 10, xxx\dots x$ ，表示下溢。浮点数值趋于零，机器不做溢出处理，而是按机器零处理。



2. 浮点乘除运算

$$x = M_x \cdot 2^{E_x} \quad y = M_y \cdot 2^{E_y}$$

1. 乘法

$$x \cdot y = (M_x \cdot M_y) \times 2^{E_x + E_y}$$

2. 除法

$$\frac{x}{y} = \frac{M_x}{M_y} \times 2^{E_x - E_y}$$

3. 步骤

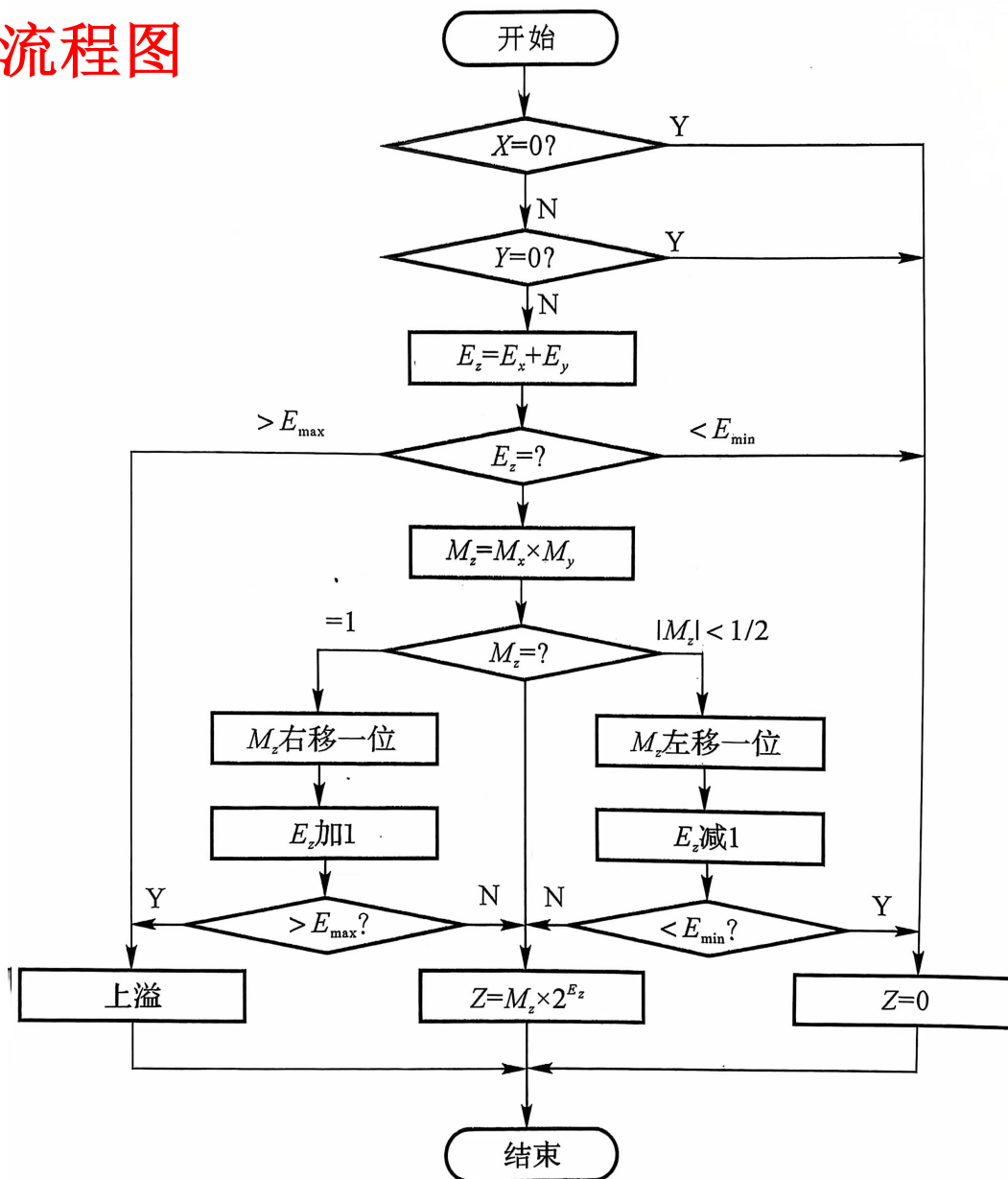
- (1) 阶码采用 补码定点加（乘法） 减（除法） 运算
- (2) 尾数乘除同 定点 运算
- (3) 规格化

4. 浮点运算部件： 阶码运算部件， 尾数运算部件



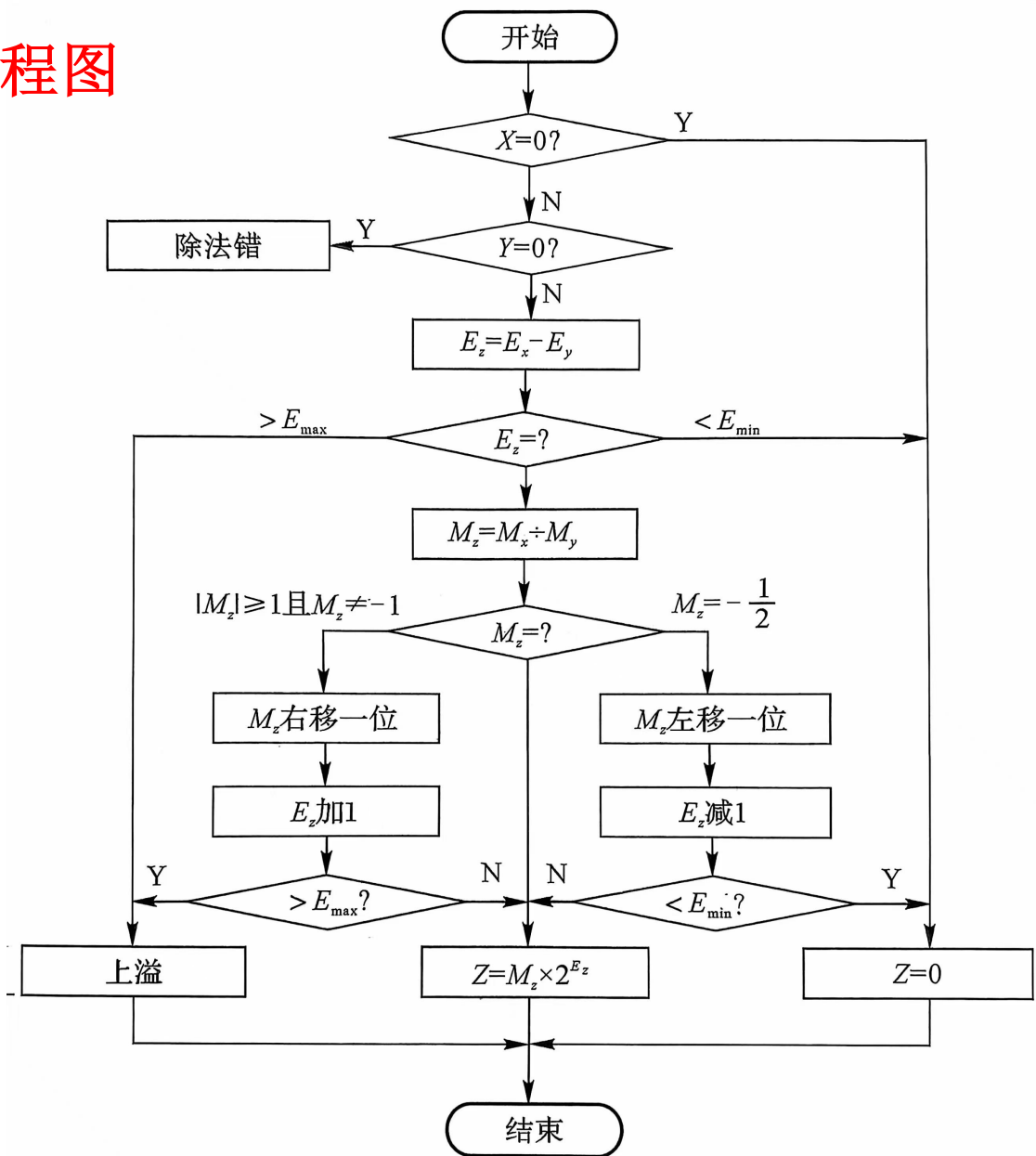
四. 浮点数运算

浮点乘法运算流程图





浮点除法运算流程图





四. 浮点数运算

[例3.18] 一浮点数表示格式为：10位浮点数，阶码4位，包含1位阶符，用移码表示，尾数6位，包含1位数符，用补码表示，阶码在前，尾数（包含数符）在后。已知： $X = (-0.11001) \times 2^{011}$ ， $Y = 0.10011 \times 2^{-001}$ ，求 $Z = X \cdot Y$ 。要求阶码用移码表示，尾数用Booth算法计算。

解：按照浮点数的格式分别写出它们的表示形式，为计算方便，阶码采用双符号位移码，尾数采用双符号位补码：

$$[X]_{\text{浮}} = 01\ 011\ 11.00111; \quad [Y]_{\text{浮}} = 00\ 111\ 00.10011;$$

移码的双符号表示： $[X]_{\text{移}} = 2^n + [X]_{\text{补}} \pmod{2^{n+1}}$

当结果的最高符号位 $SF_1 = 1$ 时表示结果溢出， $SF_1 = 0$ 时表示结果正确。

$SF_1 SF_2 = 1\ 0$ 时，结果正溢出；

$SF_1 SF_2 = 1\ 1$ 时，结果负溢出。

$SF_1 SF_2 = 0\ 1$ 时，结果为正；

$SF_1 SF_2 = 0\ 0$ 时，结果为负；



四. 浮点数运算

$$[X]_{\text{浮}} = \mathbf{01\ 011\ 11.00111}; \quad [Y]_{\text{浮}} = \mathbf{00\ 111\ 00.10011};$$

(1) 阶码相加

$$[E_Z]_{\text{移}} = [E_X]_{\text{移}} + [E_Y]_{\text{补}} = 01\ 011 + 11\ 111 = 01\ 010, \text{ 结果无溢出。}$$

$$[E_Z]_{\text{移}} = 1\ 010$$

$$[E_Z]_{\text{移}} = [E_x + E_y]_{\text{移}} = [E_x]_{\text{移}} + [E_y]_{\text{补}}$$

$$[E_Z]_{\text{移}} = [E_x - E_y]_{\text{移}} = [E_x]_{\text{移}} + [-E_y]_{\text{补}}$$

(2) 尾数相乘

采用Booth算法计算 $[M_X \cdot M_Y]_{\text{补}}$ ，首先写出下列数据

$$[M_X]_{\text{补}} = 11.00111; \quad [M_Y]_{\text{补}} = 0.10011;$$

$$[-M_X]_{\text{补}} = 00.11001$$

$$\because [X]_{\text{移}} = 2^n + X, \quad -2^n \leq X < 2^n$$

$$[Y]_{\text{移}} = 2^n + Y, \quad -2^n \leq Y < 2^n$$

$$\begin{aligned} \therefore [X]_{\text{移}} + [Y]_{\text{移}} &= 2^n + X + 2^n + Y \\ &= 2^n + [2^n + (X + Y)] \\ &= 2^n + [X+Y]_{\text{移}} \end{aligned}$$

$$\text{又} \because [Y]_{\text{补}} = 2^{n+1} + Y \pmod{2^{n+1}}$$

$$\begin{aligned} \therefore [X]_{\text{移}} + [Y]_{\text{补}} &= 2^n + X + 2^{n+1} + Y \\ &= 2^{n+1} + [2^n + (X + Y)] \\ &= [X+Y]_{\text{移}} \pmod{2^{n+1}} \end{aligned}$$





$[M_Y]_{\text{补}} = 0.10011;$

$[M_X]_{\text{补}} = 11.00111;$

$[-M_X]_{\text{补}} = 00.11001$

$[M_Z]_{\text{补}} = 1.10001\ 00101$

部分积	乘数Y ($Y_n Y_{n+1}$)	操作说明
00.00000	0.1 0 0 1 <u>1</u> 0	$Y_5 Y_6 = 10$, $+[-X]_{\text{补}}$
+ 00.11001		
00.11001	1 0.1 0 0 <u>1</u> 1	右移一位
00.01100		$Y_4 Y_5 = 11$, $+0$
+ 00.00000		
00.01100	0 1 0.1 0 <u>0</u> 1	右移一位
00.00110		$Y_3 Y_4 = 01$, $+ [X]_{\text{补}}$
+ 11.00111		
11.01101	1 0 1 0.1 <u>0</u> 0	右移一位
11.10110		$Y_2 Y_3 = 00$, $+0$
+ 00.00000		
11.10110	0 1 0 1 0. <u>1</u> 0	右移一位
11.11011		$Y_1 Y_2 = 10$, $+ [-X]_{\text{补}}$
+ 00.11001		
00.10100	0 0 1 0 1 <u>0</u> . 1	右移一位
00.01010		$Y_0 Y_1 = 01$, $+ [X]_{\text{补}}$
+ 11.00111		
11.10001	0 0 1 0 1	



四. 浮点数运算

$$[M_Z]_{\text{补}} = 1.10001\ 00101$$

(3) 结果规格化

尾数的绝对值小于 2^{-1} , 尾数为
双符号补码 $11.1XX\dots X$ 或者是
 $00.0XX\dots X$

$$M_Z \text{左规1次得: } [M_Z]_{\text{补}} = 1.00010\ 0101\mathbf{0}$$

$$E_Z \text{减1 (+} [-1]_{\text{补}} \text{)} \text{得: } [E_Z]_{\text{移}} = 01\ 010 + 11\ 111 = 01\ 001$$

(4) 舍入

$$\text{对尾数 } M_Z \text{进行 0 舍 1 入, 最后得 } [Z]_{\text{浮}} = 1\ 001\ 1.00010$$



四. 浮点数运算

[例3.19]一浮点数表示格式为：10位浮点数，阶码4位，包含1位阶符，用移码表示，尾数6位，包含1位数符，用补码表示，阶码在前，尾数（包含数符）在后。已知： $X = (-0.11001) \times 2^{011}$ ， $Y = 0.10011 \times 2^{-001}$ ，求 $Z = X \div Y$ 。要求阶码用移码计算，尾数用原码加减交替除法计算。

解：按照浮点数的格式分别写出它们的表示形式：

$$[X]_{\text{浮}} = 1 \ 011 \ 1.00111; \quad [Y]_{\text{浮}} = 0 \ 111 \ 0.10011;$$



四. 浮点数运算

$$[X]_{\text{浮}} = 1\ 011\ 1.00111; \quad [Y]_{\text{浮}} = 0\ 111\ 0.10011;$$

(1) 阶码相减 $[E_y]_{\text{移}} = 2^n + [E_y]_{\text{补}} \pmod{2^{n+1}}$

$$[E_z]_{\text{移}} = [E_x]_{\text{移}} + [-E_y]_{\text{补}} = 01\ 011 + 00\ 001 = 01\ 100$$

(2) 尾数相乘

采用原码加减交替法计算 $|M_x| \div |M_y|$ ，首先写出下列数据

$$|M_x| = 00.11001;$$

$$|M_y| = 00.10011;$$

$$[-M_y]_{\text{补}} = 11.01101$$



$$|M_X| = 00.11001;$$

$$|M_Y| = 00.10011;$$

$$[-M_Y]_{\text{补}} = 11.01101$$

$$|M_Z| = |M_X| \div |M_Y| = 1.01010$$

被除数 余数	商Q	操作说明
00.11001	0 0 0 0 0 0	
+ 11.01101		$+ [- M_Y]_{\text{补}}$
00.00110	0 0 0 0 0 1	$R_0 > 0$, 上商1
00.01100	0 0 0 0 0 1.0	左移一位
+ 11.01101		$+ [- M_Y]_{\text{补}}$
11.11001	0 0 0 0 1.0	$R_1 < 0$, 上商0
11.10010	0 0 0 1.0 0	左移一位
+ 00.10011		$+ M_Y $
00.00101	0 0 0 1.0 1	$R_2 > 0$, 上商1
00.01010	0 0 1.0 1 0	左移一位
+ 11.01101		$+ [- M_Y]_{\text{补}}$
11.10111	0 0 1.0 1 0	$R_3 < 0$, 上商0
11.01110	0 1.0 1 0 0	左移一位
+ 00.10011		$+ M_Y $
00.00001	0 1.0 1 0 1	$R_4 > 0$, 上商1
00.00010	1.0 1 0 1 0	左移一位
+ 11.01101		$+ [- M_Y]_{\text{补}}$
11.01111	1.0 1 0 1 0	$R_5 < 0$, 上商0
+ 00.10011		$+ M_Y $ 恢复余数
00.00010		



$$|M_Z| = |M_X| \div |M_Y| = 1.01010$$

(3) 结果规格化

$\because |M_X| > |M_Y|$, $\therefore |M_Z| > 1$, 必须右规1位, 得 $|M_Z| = 0.10101\ 0$

E_Z 加1得: $[E_Z]_{\text{移}} = 01\ 100 + 00\ 001 = 01\ 101$

(4) 舍入

对 $|M_Z|$ 进行 0 舍 1 入, 得 $|M_Z| = 0.10101$, $[M_Z]_{\text{原}} = 1.10101$

$$[M_Z]_{\text{补}} = 1.01011$$

最后: $[Z]_{\text{浮}} = 1\ 101\ 1.01011$



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器
2. 浮点运算器



一. 逻辑与移位运算

1. 机器数的逻辑运算
2. 机器数的移位运算
3. 带符号数的舍入操作

二. 定点数加减运算

1. 补码加减法运算
2. 补码的溢出判断与检出方法
3. 加减法的逻辑实现
4. 原码加减法运算*
5. 移码加减法运算*

三. 定点数乘除运算

1. 原码一位乘法
2. 补码一位乘法

3. 阵列乘法器

4. 原码除法

5. 补码除法*

四. 浮点数运算

1. 浮点加减运算
2. 浮点乘除法运算*

五. 十进制整数的加法运算

1. BCD码加法

六. 运算器的组成与结构

1. 定点运算器

2. 浮点运算器



六. 运算器的组成与结构

- 将**逻辑运算**、**移位运算**、**各种算术运算**的逻辑实现集成在一起就可以构成**CPU中的运算器**。
- 运算器可分为定点运算部件和浮点运算部件。
 - **定点运算部件**又称为**算术逻辑运算单元 (Arithmetic Logic Unit, ALU)**，可以进行定点数据的逻辑、移位、算术运算。
 - **浮点运算部件 (Float Point Unit, FPU)** 负责进行浮点数的算术运算。浮点运算相对定点运算要复杂得多，早期通过单独的浮点处理芯片实现，目前大多集成在**CPU内部**。



1. 定点运算器

各种计算机中定点运算器一般包含如下几个基本部分：

- ①算术逻辑运算单元
- ②通用寄存器组
- ③输入数据选择电路
- ④输出数据控制电路

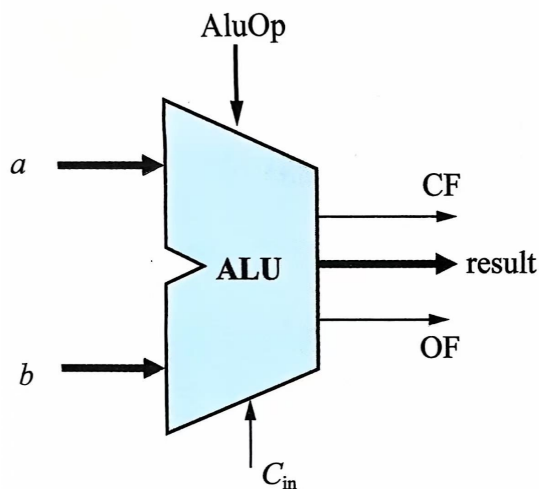


六. 运算器的组成与结构

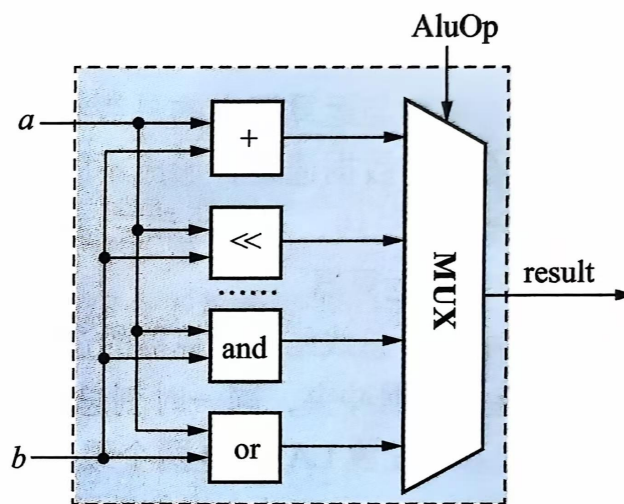
(1) 运算器组成

① 算术逻辑运算单元 (ALU)

- 是对**定点数据**进行加工处理的**纯组合逻辑电路**
- 主要功能包括**算术运算和逻辑运算**，也常作为**数据传送的通路**
- ALU的封装及内部原理如图所示。



(a)



(b)

AluOp为运算功能选择操作码，用于选择ALU内部的运算电路。



六. 运算器的组成与结构

- ALU的输出还包括若干状态标志位，常见的状态标志位如下。
 - **CF (Carry Out Flag)**：表示加法进位输出、减法借位输出或逻辑左移操作的溢出位。
 - **ZF (Zero Flag)**：为1表示运算结果为0。
 - **SF (Sign Flag)**：为1表示运算结果为负数。
 - **OF (Overflow Flag)**：为1表示有符号运算溢出。
- 很多计算机会将这些标志位暂存在一个状态寄存器中，为后续指令提供执行依据，如**x86的EFLAGS寄存器**，x86中的条件分支指令会根据标志位的不同进行不同的操作。
- 但也有一些计算机中没有状态寄存器，如**MIPS、RISC-V**，其条件分支指令直接根据ALU当前状态标志位执行不同的分支。
- 不论哪种结构，**ALU都会产生这些标志**，只是不同计算机利用这些标志的方法和时机不同而已。



六. 运算器的组成与结构

② 通用寄存器组

通用寄存器组也称为通用寄存器堆，包括一组通用寄存器，寄存器的作用大致可分为以下3类。

- 暂时存放参加运算的数据和运算结果，尽量减少指令执行过程中访问主存的次数，以提高运算速度。
- 作为状态寄存器，保存运算过程中设置的状态，如进位、溢出、结果为负等。这些状态可用于程序执行流程的控制。
- 可作为变址寄存器、堆栈指示器使用。不同的计算机对这组寄存器的使用情况和设置个数不相同。



六. 运算器的组成与结构

③ 输入数据选择控制

- **输入数据选择控制**是指对送入ALU的数据进行选择和控制。

④ 输出数据控制电路

- **输出数据控制电路**对ALU输出的数据进行**控制**，该电路一般还具有**移位**功能，并**可将ALU输出的数据输送到ALU输入端、通用寄存器的通路，以及送往总线的控制电路。**

⑤ 内部总线

- 内部总线是连接各个ALU、通用寄存器组、缓冲寄存器等功能部件的信息通道。



六. 运算器的组成与结构

(2) 运算器的基本结构

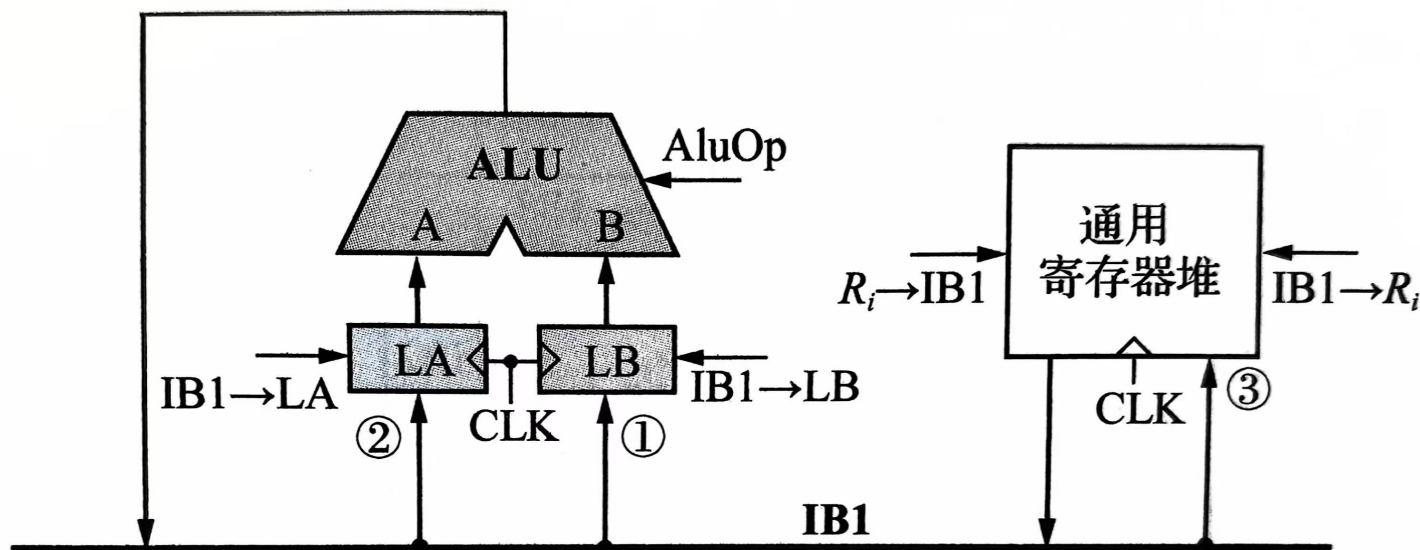
运算器的基本结构与**运算器中的总线结构**以及**运算器各部件与总线的连接方式**紧密相关，不同的连接构成不同的数据通路，形成不同结构的运算器。

根据运算器中数据通路的不同，可将运算器的结构分为单总线、双总线和三总线3种。



六. 运算器的组成与结构

①单总线结构运算器



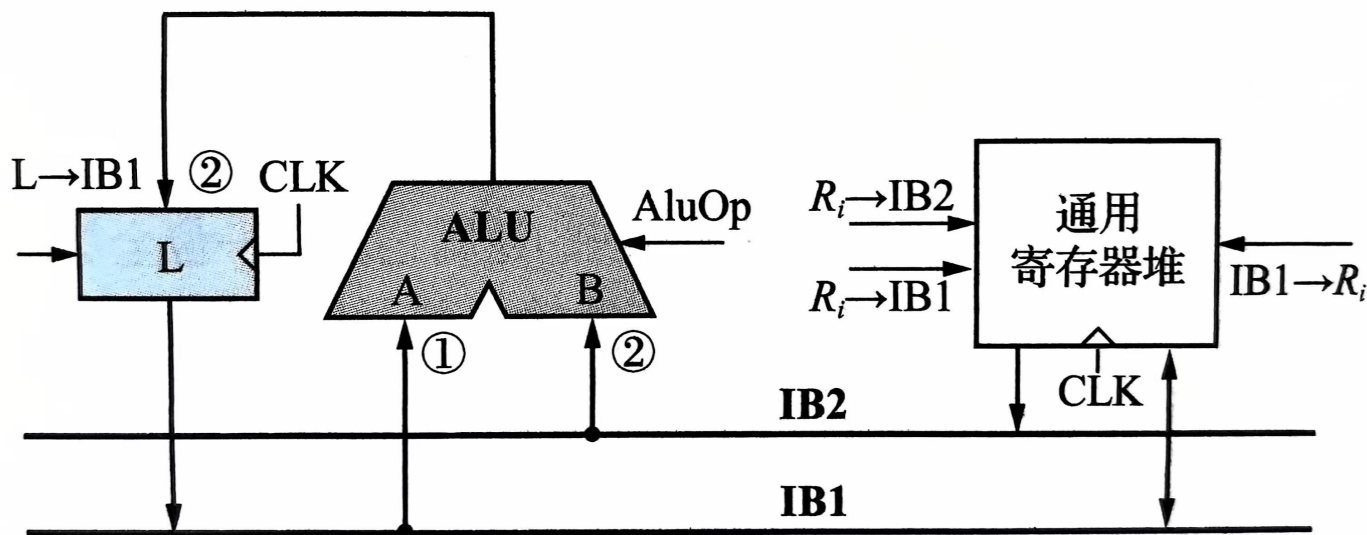
单总线结构运算器

- 所有部件都与内部总线IB1（Internal Bus）连接。
- 为避免数据冲突，同一时刻总线上只能传输一个数据，为此需要在ALU输入端设置LA、LB两个缓冲寄存器。
- 运算结果可通过总线存入通用寄存器或缓冲器LA或LB中。



六. 运算器的组成与结构

② 双总线结构运算器



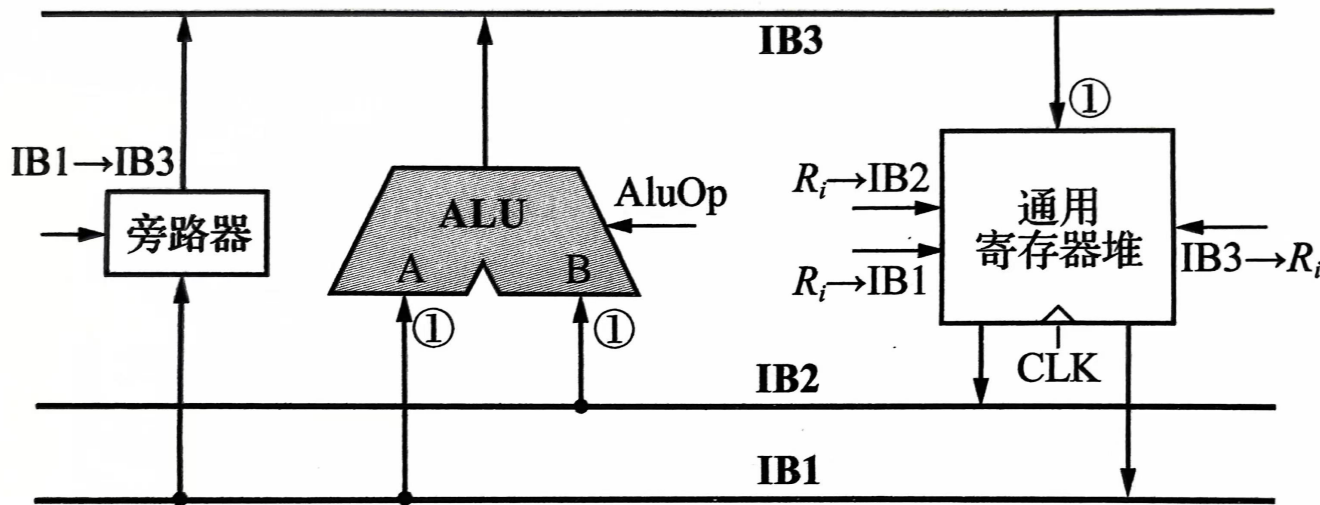
双总线结构运算器

- ALU与通用寄存器堆都连接在总线IB1和IB2上，通用寄存器堆有两个输出端口，可以通过两组总线分别将两个寄存器操作数同时加载到ALU的两个输入端。
- 为防止ALU的输出结果直接送入总线IB1而产生数据冲突，在ALU的输出与IB1总线间设置了缓冲寄存器L暂存运算结果。除缓冲功能外，缓冲寄存器L往往还具备数据移位功能。



六. 运算器的组成与结构

③ 三总线结构运算器



三总线结构运算器

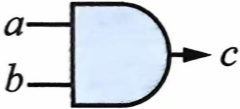
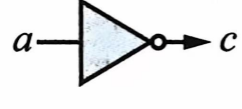
- 在执行双操作数运算时，可同时通过3组总线传输数据。
- 旁路器的作用是不通过ALU实现通用寄存器之间的数据传输。
- 三总线结构可以同时给出 $R_i \rightarrow IB1$ 、 $R_i \rightarrow IB2$ 、 $AluOp$ 、 $IB3 \rightarrow R_i$ 信号，在时钟周期配合下完成运算，整个运算只需要个时钟周期，速度是3种结构中最快的，且不需要缓冲寄存器，但其通用寄存器堆需要提供提供一个读端口、一个写端口。



六. 运算器的组成与结构

(3) ALU设计

ALU是计算机的核心部件，能实现的基本功能包括加、减等算术运算和与、或、非等逻辑运算。实现上述算术运算和逻辑运算功能的部件就是构造ALU的基本单元，它们的逻辑符号和基本功能如下图所示。

与门	或门	非门	异或门	全加器	多路选择器
					
$c = a \& b$	$c = a b$	$c = \sim a$	$c = a \wedge b$	$c = a + b$	$c = (s == 0) : a, b$

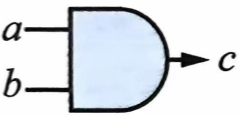
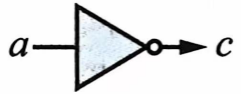
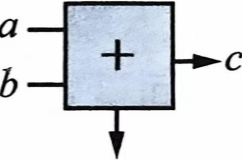
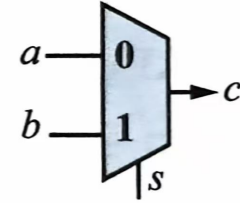
构造ALU的基本单元



六. 运算器的组成与结构

(3) ALU设计

ALU是计算机的核心部件，能实现的基本功能包括加、减等算术运算和与、或、非等逻辑运算。实现上述算术运算和逻辑运算功能的部件就是构造ALU的基本单元，它们的逻辑符号和基本功能如下图所示。

与门	或门	非门	异或门	全加器	多路选择器
					
$c = a \& b$	$c = a b$	$c = \sim a$	$c = a \wedge b$	$c = a + b$	$c = (s == 0) : a, b$

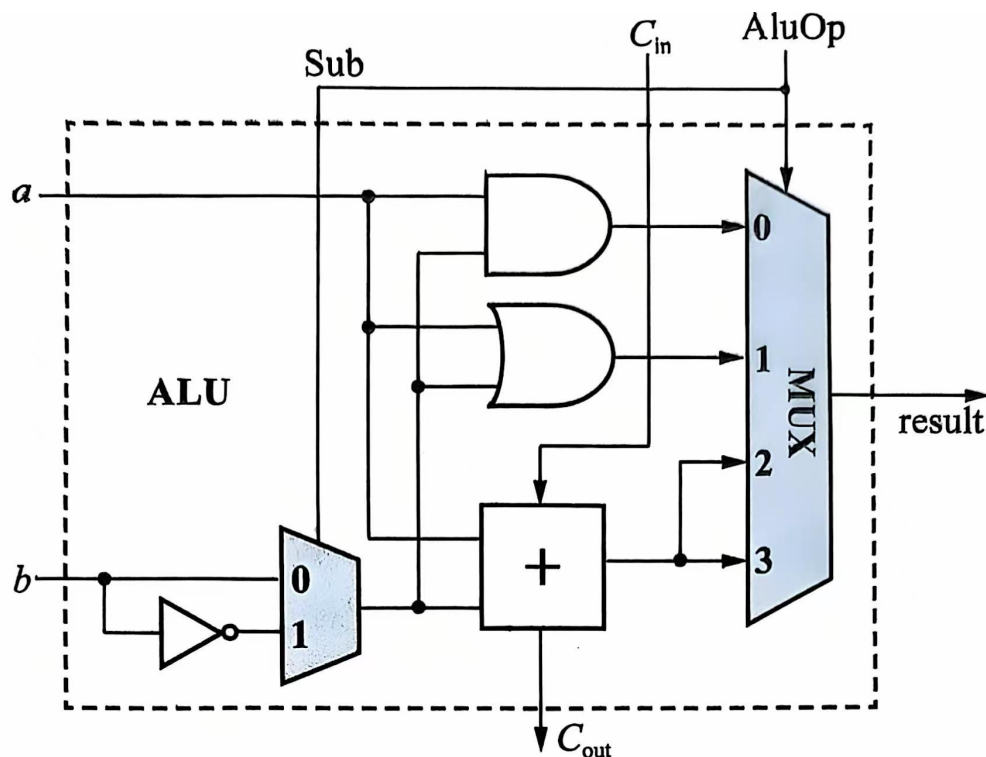
构造ALU的基本单元



六. 运算器的组成与结构

(3) ALU设计

下图所示是一个利用上述算术、逻辑单元构建的一位ALU。



基本的一位ALU

图中AluOp为2位运算选择码

- 当 $\text{AluOp}=0$ 时，ALU完成逻辑与运算
- 当 $\text{AluOp}=1$ 时，完成逻辑或运算
- 当 $\text{AluOp}=2$ 时，Sub信号应通过相关逻辑译码为0。通过二路选择器选择b进入全加器，运算结果为 $a+b$
- 当 $\text{AluOp}=3$ 时，如果将Sub信号连接到 C_{in} 端，ALU可以完成减法运算。



将多个1位ALU按进位链串联即可得到n位的ALU。



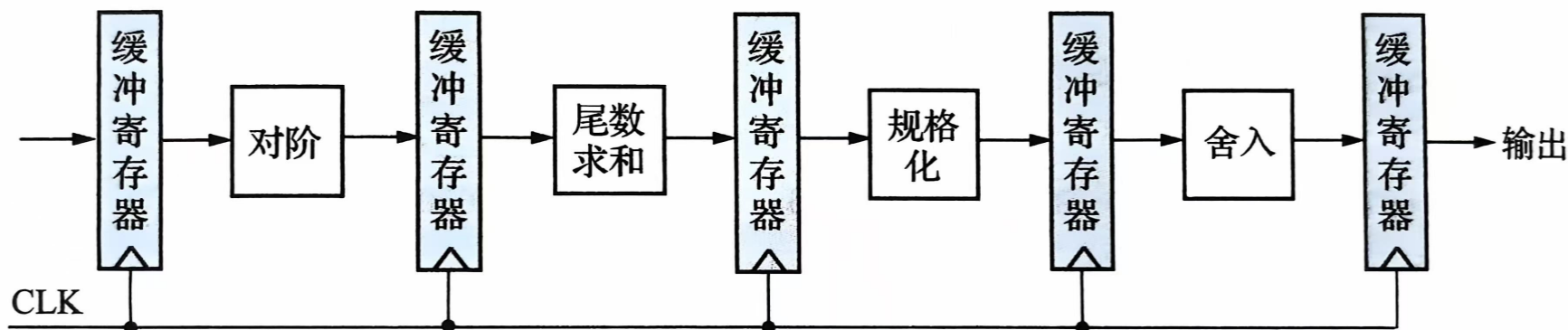
- 进行多位减法时，**最低位进位位应该置1**，也就是要将Sub信号连接到低位ALU的进位输入端，**实现末位加1的功能**。
- 串行进位电路性能较差，可采用前面介绍的快速先行进位电路对其进行性能优化



六. 运算器的组成与结构

2. 浮点运算器

- 浮点数据采用定点数据进行表示，运算操作数单独存放在浮点寄存器中，浮点数的运算过程相对较为复杂，速度较慢。
 - 以加减法为例，计算过程包括对阶、尾数运算、结果规格化、舍入处理、溢出判断等多个步骤，每一个步骤都由相应的组合逻辑电路进行实现，将每个步骤的运算逻辑电路串联在一起即可构成浮点加减法电路。
- 但这样的浮点运算器时间延迟过长，为进一步优化性能，可以按照乘法流水线的思路将运算过程流水化。



浮点数加减法流水线框图



西安电子科技大学
XIDIAN UNIVERSITY

THE END !

THANKS